第12章: テストの書き方と実行 — 壊していないかを自動で確かめる（完全版）

> この章では、**プログラムが正しく動くことを「自動で」確かめるしくみ＝テスト** を学びます。あなたが画面の文字や色を1つ直しただけのつもりでも、思わぬ場所が壊れることがあります。毎回すべての画面を手で操作して確認するのは大変ですし、見落としも起きます。そこで **テストコード**（テスト用のプログラム）を書いておき、コマンド1つで「全部ちゃんと動くか」を機械に確認させます。
>
> 本アプリには、フロント（React/TypeScript）のテストと、バック（Django/Python）のテストの **両方** が用意されています。この章では、実際のテストファイル `Login.test.tsx`（フロント・全132行）と `test_backend.py`（バック）を **1行ずつ** 読み解き、テストの「準備→実行→検証」の3ステップを体に染み込ませます。
>
> この章を読み終えると、「修正したあと `npm run test:run` と `python manage.py test` を流して、全部通れば安心して提出できる」という、**プロの保守の最後の砦** が使えるようになります。

この章で学ぶこと

- **テストとは何か・なぜ書くのか** — 健康診断のたとえ。手作業確認との違い
- **テストの3ステップ** — 準備（Arrange）→実行（Act）→検証（Assert）
- **フロントのテスト** — Vitest（ヴィテスト） ＋ React Testing Library のしくみ
- `Login.test.tsx` **全132行を1行ずつ完全解説** — `vi.mock` によるモック化、`describe` / `it` / `beforeEach`、`render` / `screen` / `fireEvent` / `waitFor` / `findBy`、`getByRole` / `getByLabelText`、`mockResolvedValueOnce` / `mockRejectedValueOnce`、成功遷移・エラー表示・401遷移の各テスト
- `getBy` / `findBy` / `queryBy` **の使い分け** — いつどれを使うか
- **バックのテスト** — Django `TestCase` ＋ `APIClient`（エーピーアイクライアント）
- `test_backend.py` **の冒頭〜** `setUp` **を1行ずつ解説** — `make_break_times`・`BaseTestCase`・`login` ヘルパー
- `TestModels` **と各テストの代表例を解説** — `assertEqual` / `assertIn` / `assertTrue` 等
- **APIの 401/403/200 を確認するテストの考え方** — 認証・権限のテスト
- **実行コマンドと出力例** — `npm run test:run` / `python manage.py test --settings=hozen_another.test_settings`
- **テストが失敗したときの読み方と対応、新しいテストの足し方**

目次

0. 前提知識：テストとは何か、なぜ書くのか

1. テストの3ステップ（準備→実行→検証）
2. フロントのテスト道具：Vitest と React Testing Library
3. モック（にせもの）という考え方
4. 実コード全解説：Login.test.tsx 全132行
5. getBy / findBy / queryBy の使い分け
6. フロントのテスト設定ファイル
7. バックのテスト道具：Django TestCase と APIClient
8. 実コード全解説：test_backend.py の土台（冒頭〜setUp）
9. 実コード全解説：TestModels（モデルのテスト）
10. APIの 401 / 403 / 200 を確かめるテストの考え方
11. assertXXX メソッド早見表
12. バックのテスト設定（test_settings.py）
13. テストの実行コマンドと出力例
14. テストが失敗したときの読み方と対応
15. 新しいテストの足し方
16. トラブルシューティング
17. 演習問題
18. この章のまとめ

---

0. 前提知識：テストとは何か、なぜ書くのか

0.1 テストとは「機械にやらせる動作確認」

> **テスト（test）とは？** プログラムが「期待どおりに動くか」を確かめるための、**別のプログラム**。テスト対象の本体コードを呼び出し、「こういう入力を与えたら、こういう結果になるはず」を機械的にチェックします。

たとえば、本アプリのログイン機能には「従業員番号 `12345` を送ったらログイン成功する」という期待があります。これを手で確認するなら、毎回ブラウザを開いて、番号を打って、ボタンを押して、画面を見る……という作業が要ります。テストコードを書いておけば、**コマンド1つでこの確認を一瞬で・何百回でも** やってくれます。

> ▼ **たとえ話：健康診断** テストは、人間の **健康診断** に似ています。体のあちこち（血圧・血液・視力…）を一通り測り、「基準値の範囲内か」を確認します。プログラムも同じで、機能のあちこちに「測定項目（テスト項目）」を置いておき、コマンドを流すたびに **全項目を一斉に検査** します。どこか1つでも基準を外れたら、テストは「ここが異常です」と赤字で教えてくれます。
>
> 健康診断を毎年受けるように、テストも **コードを直すたびに** 流します。そうすれば「気づかないうちに体（プログラム）の別の場所が悪くなっていた」を早期発見できます。

0.2 なぜ手作業ではなくテストコードなのか

確認方法	かかる時間	見落とし	何百回の繰り返し
手作業（毎回ブラウザを操作）	1機能で数分、全機能で数時間	起きやすい（人間だから）	苦痛・現実的でない
テストコード（コマンド1つ）	全部で数秒〜数十秒	起きにくい（毎回同じ手順）	何度でも一瞬

> **なぜ？ テストが「保守」で最も効く理由** あなたは後任者として、これからこのアプリを **直し続けます**。1か所直すと、関係なさそうな別の場所が連鎖的に壊れること（これを **デグレード／リグレッション＝後退バグ** と呼びます）がよくあります。テストがあれば、修正後にコマンドを流すだけで「壊していないか」が即わかります。**テストは「安心して直すための保険」** です。

0.3 本アプリのテストは2種類ある

種類	対象	道具	場所	実行コマンド
フロントのテスト	React の画面・部品	Vitest ＋ React Testing Library	`frontend/src/tests/`	`npm run test:run`
バックのテスト	Django のAPI・モデル・関数	Django `TestCase` ＋ `APIClient`	`kosu/tests/`	`python manage.py test ...`

この章では、両方を実物のコードで読みます。まずは「どんなテストでも共通の考え方」である **3ステップ** を押さえましょう。

---

1. テストの3ステップ（準備→実行→検証）

ほぼすべてのテストは、次の3つの段階でできています。英語の頭文字をとって **AAA（トリプルA）** とも呼ばれます。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
1. 準備（Arrange / アレンジ）  … テストの前提を整える
                                （データを作る・にせものを用意する・画面を描く）
2. 実行（Act / アクト）        … テストしたい動作を1回だけ行う
                                （ボタンを押す・関数を呼ぶ・APIを叩く）
3. 検証（Assert / アサート）   … 結果が期待どおりかを確かめる
                                （「〜のはず」をチェックし、違えば失敗）
```

> **用語: アサーション（assertion）**「〜であるはずだ」という **主張（断言）** のこと。テストの心臓です。`expect(x).toBe(5)`（フロント）や `self.assertEqual(x, 5)`（バック）のように書き、もし実際の値が主張と違えば、その場でテストは「失敗」になります。

1.1 3ステップを日本語の例で

> ※説明用の簡易例（本アプリのコードではありません）

コピー

```
【準備】  電卓アプリを開く
【実行】  「2」「＋」「3」「＝」を押す
【検証】  画面に「5」が表示されているはず → 違えば失敗
```

このたった3段で、1つのテストが完成します。`Login.test.tsx` も `test_backend.py` も、すべてこの形の繰り返しです。**「いまどのステップを読んでいるか」を意識すると、テストコードが一気に読みやすくなります。**

---

2. フロントのテスト道具：Vitest と React Testing Library

フロントのテストは、2つの道具を組み合わせて書きます。

> **Vitest（ヴィテスト）とは？** JavaScript/TypeScript 用の **テスト実行ツール**（テストランナー）。テストファイルを探して実行し、結果（成功/失敗）をまとめて表示してくれます。`describe`（グループ分け）・`it`（1つのテスト）・`expect`（検証）といった「テストを書くための言葉」を提供します。Vite（ヴィート）というビルドツールと相性がよく、速いのが特徴です。
>
> **React Testing Library（リアクト・テスティング・ライブラリ。略してRTL）とは？** React の画面を **テスト用に仮想で描き**、「画面に○○の文字があるか」「ボタンを押したらどうなるか」を、**ユーザーの視点で** 確認するためのライブラリ。内部の細かい実装ではなく「**人が見て・触って分かること**」を確認するのが思想です。`render`（描く）・`screen`（画面）・`fireEvent`（操作する）などを提供します。
>
> **なぜ「ユーザーの視点」が大事？**「このボタンの内部関数が呼ばれたか」ではなく「ボタンを押したらエラーが画面に出たか」を確認する方が、**実際の使われ方に近く、壊れに強い** テストになります。内部の作りを変えても、ユーザーから見た振る舞いが同じなら、テストは通り続けるからです。

2.1 jsdom ―「ブラウザのふり」をする道具

テストはブラウザではなく **コマンドライン（黒い画面）** で動きます。でも React は本来ブラウザの画面（DOM）を相手にします。そこで **jsdom（ジェイエスドム）** という「ブラウザの画面のふりをするライブラリ」を使い、コマンドライン上に **にせの画面** を用意します。これにより、ブラウザを起動しなくても画面のテストができます（設定は§6で実コードを見ます）。

---

3. モック（にせもの）という考え方

`Login.test.tsx` を読む前に、最重要キーワード **モック** を理解します。

> **モック（mock）とは？** テスト対象が依存している「外部のもの」を、**にせもの（替え玉）に差し替える** こと。本物の代わりに、テスト都合のいいように動く偽物を使います。「モック化する」「モックする」と言います。

3.1 なぜモックが必要か

ログイン画面 `Login.tsx` は、内部で次の3つに **依存** しています。

1. **API通信**（`api.post(...)` でサーバーへ送る）
2. **画面遷移**（`useNavigate()` で別ページへ移動する）
3. **ロゴ画像**（`import logo from "../../img/TitleRogo.png"`）

テストのときにこれらを本物のまま動かすと、困ったことが起きます。

依存	本物だと困ること	モックにすると
API通信	テストのたびに本物のサーバーに繋ぐ必要があり、結果も毎回変わる	「成功を返す/失敗を返す」を **テストが自由に決められる**
画面遷移	コマンドラインには本物のブラウザ遷移がない	「遷移しようとしたか・どこへか」を **記録だけする** にせ関数にできる
画像	テスト環境では画像ファイルを読み込めない	「`logo.png` という文字列」に置き換えてエラーを避ける

> ▼ **たとえ話：映画の撮影** モックは、映画撮影の **スタント（替え玉）や小道具** に似ています。本物の崖から飛び降りる代わりにスタントを使い、本物のお金の代わりに小道具のお札を使います。テストでも「本物だと危険・不便・不安定なもの」を、安全で都合のいいにせものに替えるのです。

3.2 Vitest でモックする道具

書き方	意味
`vi.fn()`	**にせの関数**（モック関数）を作る。呼ばれた回数や引数を記録してくれる
`vi.mock("モジュール名", () => ...)`	あるモジュール（ファイル）丸ごとを、にせものに差し替える
`xxx.mockResolvedValueOnce(値)`	「次の1回だけ、**成功**してこの値を返す」ように仕込む（API成功を演じる）
`xxx.mockRejectedValueOnce(値)`	「次の1回だけ、**失敗**してこのエラーを投げる」ように仕込む（API失敗を演じる）
`vi.clearAllMocks()`	モックの記録（呼ばれた回数など）を **全部リセット** する

これだけ頭に入れたら、いよいよ実コードを読みます。

---

4. 実コード全解説：Login.test.tsx 全132行

ここが本章フロント編の核心です。本アプリのログイン画面のテスト `frontend/src/tests/MainPage/Login.test.tsx` を、**1行残らず** 読みます。このファイルには「成功・失敗・401遷移・別エラー」という4つの代表テストが詰まっており、**フロントのテストの型がすべて入っています。**

4.1 インポート部（1〜5行目）

> **▼ このコードがやること（先に日本語で）**：テストを書くのに必要な道具を、各ライブラリから読み込みます。Vitest の言葉（describe など）、RTL の道具（render など）、ブラウザのふりをする道具、そしてテスト対象の `Login` 画面本体を取り込みます。

コピー

```
// ① Vitestの道具を読み込む
import { vi, describe, it, expect, beforeEach } from "vitest";
// ② RTLの道具を読み込む
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
// ③ テスト用のルーター（後述）
import { MemoryRouter } from "react-router-dom";
// ④ 「画面にooがある」を確認する追加機能
import "@testing-library/jest-dom/vitest";
// ⑤ テスト対象＝ログイン画面本体
import Login from "../../MainPage/Login";
```

> **▶ ここで初めて出てくるコード**
>
> - `MemoryRouter`（Vitest・Testing Library）：React Router の部品で、URL をブラウザーではなくメモリーの中で持つ物。テストの中で `<MemoryRouter>` で囲むと、`Link` や `useNavigate` を使う部品をそのまま試せます。

1行ずつ見ます。

- ① `import { vi, describe, it, expect, beforeEach } from "vitest";` Vitest から5つの道具を取り出します。
- `vi` … モックを作る道具（§3）。
- `describe` … テストを **グループにまとめる** 箱。
- `it` … **1つのテスト** を書く単位（「〜であること」と読む）。
- `expect` … **検証**（アサーション）を書く道具。
- `beforeEach` … 各テストの **前に毎回** 実行する準備処理。
- ② `import { render, screen, fireEvent, waitFor } from "@testing-library/react";` RTL から4つの道具を取り出します。
- `render` … React の画面を **テスト用に描く**（仮想画面に表示する）。
- `screen` … 描かれた **画面**（**仮想DOM**）**全体** を表すオブジェクト。ここから要素を探します。
- `fireEvent` … クリックや入力など、**ユーザー操作を発火** させる道具。
- `waitFor` …「ある条件が満たされるまで **待つ**」道具（非同期処理の完了待ち）。

- ③ `import { MemoryRouter } from "react-router-dom";` `MemoryRouter`（メモリールーター）は、テスト専用の **にせのルーター（経路管理）**。本物のルーターはブラウザのURLを使いますが、テストにはブラウザのURLがありません。そこで「メモリ上だけで動くルーター」で `Login` を包みます。

> **なぜ？** `Login.tsx` の中には `<Link to="/help">`（リンク）や `useNavigate()`（遷移）が入っています。これらは「ルーターの中で使う」ことが前提なので、ルーター無しで描こうとするとエラーになります。`MemoryRouter` で包むことで、ルーターがある状態を再現します。

- ④ `import "@testing-library/jest-dom/vitest";` jest-dom（ジェスト・ドム）という追加機能を読み込みます。これにより `toBeInTheDocument()`（画面に存在するか）や `toHaveTextContent()`（この文字を含むか）といった、**画面チェック専用の便利な検証** が使えるようになります。`/vitest` を付けるのは「Vitest向けに登録する」という意味です。

- ⑤ `import Login from "../../MainPage/Login";` **テストの主役**、ログイン画面本体を読み込みます。`../../` は「2つ上のフォルダ」。テストファイルは `src/tests/MainPage/` にあり、本体は `src/MainPage/` にあるので、2階層上がって `MainPage/Login` を指します。

4.2 画像のモック（7〜8行目）

> **▼ このコードがやること:** ロゴ画像の読み込みを、にせもの（ただの文字列 `"logo.png"`）に差し替えます。テスト環境では本物の画像ファイルを処理できないため、エラーを防ぐ目的です。

コピー

```
// モック設定
// 画像 import を文字列に置き換える
vi.mock("../../img/TitleRogo.png", () => ({ default: "logo.png" }));
```

> **▶ ここで初めて出てくるコード**
>
> - `.mock()`（Vitest・Testing Library）：`vi.mock("部品の場所", () => ({ … }))` は、その部品を、テスト用のにせ物（モック）に取り替えます。本物のサーバーや画像を使わずに試せます。

- `vi.mock("../../img/TitleRogo.png", () => ({ default: "logo.png" }));`「`Login.tsx` が `import logo from "../../img/TitleRogo.png"` で画像を読み込もうとしたら、**本物の画像の代わりに `{ default: "logo.png" }` を渡せ**」という指示です。
- `() => ({ default: "logo.png" })` …「`default` という名前で `"logo.png"` を返す」関数。
- `import logo from "..."` の `logo` には、この `default` の値（＝文字列 `"logo.png"`）が入ります。

なぜ `default` なの？ `import 名前 from "..."`（デフォルトインポート）で受け取る値は、モジュールの `default` というキーに入ります。だからモックも `default` で返す必要があります。画像は本来「画像ファイルへのパス（文字列）」として扱われるので、にせの文字列で十分です。

4.3 API（axios）のモック（10〜19行目）

▼ このコードがやること: API通信を担当する `api/axios` というモジュールを、丸ごと「にせの通信」に差し替えます。get/post/put/delete の4メソッドを、それぞれ「呼ばれたことを記録するだけのにせ関数」にします。さらに、そのにせモジュールを `mockedApi` という名前で取り出して、後でテストごとに「成功を返せ／失敗を返せ」と仕込めるようにします。

コピー

```
// api/axios モジュールを丸ごと差し替え
vi.mock("../../api/axios", () => ({
  //   default エクスポート（import api from で受け取る対象）
  default: {
    //   get を にせ関数に
    get: vi.fn(),
    //   post を にせ関数に（ログインで使う主役）
    post: vi.fn(),
    //   put を にせ関数に
    put: vi.fn(),
    //   delete を にせ関数に
    delete: vi.fn(),
  },
}));
// 差し替え後の（にせの）api を取り込む
import api from "../../api/axios";
// 型を any にして mockResolvedValueOnce 等を呼べるように
const mockedApi = api as any;
```

▶ ここで初めて出てくるコード

- **`any`**（TypeScript）:「何の型でもよい」という特別な型。型の確かめが効かなくなるので、ふだんは使わず、テストでにせ物を作る時など、どうしても必要な所だけに使います。
- **`.fn()`**（Vitest・Testing Library）: `vi.fn()` は、呼ばれたかどうか・何を渡されたかを覚える、にせの関数を作ります。

1行ずつ。

- `vi.mock("../../api/axios", () => ({ default: { ... } }))` 本アプリの通信モジュール `api/axios`（`07` 章で見た axios の設定済みインスタンス）を、丸ごとにせものに置き換えます。`default` の中に4つのメソッドを用意します。
- `get: vi.fn()` / `post: vi.fn()` / `put: vi.fn()` / `delete: vi.fn()` それぞれ `vi.fn()`（にせの関数）にします。にせ関数は **呼ばれた回数・引数を記録** し、後から「何を返すか」を仕込めます。ログインで実際に使うのは `post`（従業員番号をサーバーへ送る）です。残り3つも、`Login.tsx` が間接的に読み込んでもエラーにならないよう、念のため用意しています。
- `import api from "../../api/axios";` `vi.mock` で差し替えた **にせの** `api` を取り込みます。`vi.mock` は「このファイルの中で `api/axios` を読み込むものは全部にせものになる」効果があるので、ここで取り込む `api` も自動的ににせものです。
- `const mockedApi = api as any;` にせの `api` を `mockedApi` という名前に入れ直します。`as any`（アズ・エニー）は「型チェックを一旦オフにする」TypeScriptの書き方。

> **なぜ `as any` ？** にせ関数には `mockResolvedValueOnce` という特別なメソッドがありますが、本物の `api` の型定義にはそんなメソッドはありません。型のままだと「そんなメソッド無い」と怒られるので、`as any` で型チェックを外し、自由に呼べるようにします。テストならではの割り切りです。

4.4 axios本体のモック（21〜25行目）

> ▼ **このコードがやること:** axios の `isAxiosError`（このエラーは通信由来のエラーか？を判定する関数）を、にせものに差し替えます。`Login.tsx` のエラー処理がこの関数を使うため、テストでも動くようにします。

コピー

```
// axios本体を差し替え
vi.mock("axios", () => ({
  default: {
    // 「err に response があれば通信エラー」とみなす にせ実装
    isAxiosError: vi.fn((err) => !!err.response),
  },
}));
```

- `isAxiosError: vi.fn((err) => !!err.response)` `isAxiosError` は本来「渡されたエラーが axios（通信）由来かどうか」を `true/false` で返す関数です。ここでは、にせの中身として「**`err.response` があれば `true`**」という簡単な実装を入れています。
- `err.response` … 通信エラーには「サーバーからの応答（response）」が付いています。それがあれば通信エラーとみなす、という割り切り。
- `!!`（びっくり2つ）… 値を「真偽（true/false）」に変換するイディオム。`err.response` があれば `true`、無ければ `false`。

なぜモックする？ `Login.tsx` は「APIエラーが起きたとき、それが通信エラーなら `err.response.status`（401など）を見て分岐する」処理を持っています。その分岐に使う `isAxiosError` を、テストでコントロールできるよう、にせの判定に置き換えています。後のテスト（4.10・4.11）で、わざと `response` を持つエラーを投げて「通信エラー扱い」にさせます。

4.5 画面遷移（navigate）のモック（27〜31行目）

▼ このコードがやること: 画面遷移を行う `useNavigate()` を、にせの遷移関数 `mockedNavigate` に差し替えます。これにより「`Login.tsx` がどこへ遷移しようとしたか」をテストが記録・検査できます。

コピー

```
// 遷移を記録するための にせ関数
const mockedNavigate = vi.fn();
// react-router-dom を部分的に差し替え
vi.mock("react-router-dom", async () => {
  // 本物の中身を一旦すべて取得
  const actual = await vi.importActual("react-router-dom");
  // 本物はそのまま、useNavigate だけ にせに
  return { ...actual, useNavigate: () => mockedNavigate };
});
```

▶ ここで初めて出てくるコード

- `.importActual()`（Vitest・Testing Library）：`await vi.importActual("react-router-dom")` は、モックに取り替える前の本物の部品を読みます。「一部だけにせ物にして、残りは本物」にする時に使います。

1行ずつ。

- `const mockedNavigate = vi.fn();` にせの遷移関数を作ります。`Login.tsx` が `navigate("/")` のように呼ぶと、この `mockedNavigate` が「`"/"` という引数で呼ばれた」と記録します。実際の画面遷移は起きません（記録するだけ）。
- `vi.mock("react-router-dom", async () => { ... })` ルーターのライブラリ全体を差し替えます。ただし**全部にせにすると `MemoryRouter` や `Link` まで壊れて困る**ので、工夫します。
- `const actual = await vi.importActual("react-router-dom");` `vi.importActual` は「**本物の**ライブラリの中身をそのまま読み込む」道具。本物の `react-router-dom`（`MemoryRouter`・`Link`・`useNavigate` …全部入り）を `actual` に取り出します。
- `await`（アウェイト）… この読み込みが終わるまで待つ印（`05` / `04` 章の非同期）。

- `return { ...actual, useNavigate: () => mockedNavigate };`「本物の中身（`...actual`）を全部引き継ぎつつ、`useNavigate` だけ を上書きする」という意味です。
- `...actual` … スプレッド構文（`05` 章）。本物の全機能をコピー。
- `useNavigate: () => mockedNavigate` … `useNavigate()` を呼ぶと、にせの `mockedNavigate` が返るように差し替え。

> **なぜこんな部分差し替え？** `MemoryRouter` と `Link` は **本物が必要**（画面を正しく描くため）。でも `useNavigate` だけは「遷移先を記録したい」ので、にせにしたい。だから「本物全部 ＋ useNavigate だけにせ」という合わせ技にしています。これは React のテストで頻出のテクニックです。

4.6 テストのグループと共通準備（33〜36行目）

> **▼ このコードがやること:** 「Login Component」という名前で、これから書く全テストを1つのグループにまとめます。さらに、各テストが始まる前に毎回「モックの記録をリセット」して、前のテストの影響が残らないようにします。

コピー

```
// 「Login Component」というテストグループの開始
describe("Login Component", () => {
  // 各テストの実行前に、毎回これを行う
  beforeEach(() => {
    //   モックの呼び出し記録を全部リセット
    vi.clearAllMocks();
  });
```

> **▶ ここで初めて出てくるコード**
>
> - `.clearAllMocks()`（Vitest・Testing Library）：`vi.clearAllMocks()` は、にせの関数が覚えている「呼ばれた回数・渡された値」を全部消します。テストごとにまっさらにするため、`beforeEach` の中で呼びます。
> - `describe()`（Vitest・Testing Library）：`describe("まとめの名前", () => { … })` は、関係のあるテストを 1 つのまとまりにします。
> - `beforeEach()`（Vitest・Testing Library）：`beforeEach(() => { … })` は、同じまとまりの各テストの前に、毎回動かす準備を書きます。

- `describe("Login Component", () => { ... })` `describe`（ディスクライブ）は **テストの章立て**。中に複数の `it`（テスト1件）を入れ、まとめて「Login Component のテスト群」とします。実行結果でも「Login Component」という見出しの下に各テストが並びます。
- `beforeEach(() => { ... })` `beforeEach`（ビフォアイーチ）は「**各テストの直前に毎回** 走る準備処理」。

- `vi.clearAllMocks();` すべてのモックの **記録（呼ばれた回数・引数・仕込んだ戻り値）を消去** します。

> **なぜ毎回リセット？** テストは **互いに独立** であるべきです。もし前のテストで `mockedNavigate` が呼ばれた記録が残ったままだと、次のテストで「遷移したか？」を調べたとき、前の記録に引きずられて誤判定します。毎回まっさらにすることで、各テストが「ゼロから」始まり、**順番や他テストの影響を受けない** のが保証されます。これはテストの鉄則です。

4.7 テスト1：見出しと入力欄が表示されること（38〜47行目）

最初のテストです。**準備→実行（描画）→検証** の3ステップで読みます。

> **▼ このコードがやること:** ログイン画面を描いて、「ログイン」という見出しと「従業員番号」という入力欄が、ちゃんと画面に表示されているかを確認します。一番基本の「正しく表示されるか」テストです。

コピー

```
// ① テスト名（日本語でOK）
it("ログイン見出しと従業員番号入力欄が表示されること", () => {
  // ② 【実行/準備】画面を描く
  render(
    //    ルーターで包んで
    <MemoryRouter>
      {/* ログイン画面を表示 */}
      <Login />
    </MemoryRouter>
  );

  // ③ 【検証】見出しがあるか
  expect(screen.getByRole("heading", { name: "ログイン" })).toBeInTheDocument();
  // ④ 【検証】入力欄があるか
  expect(screen.getByLabelText("従業員番号")).toBeInTheDocument();
});
```

▶ ここで初めて出てくるコード

- `.getByRole()`（Vitest・Testing Library）：役割（ボタン・見出し・リンクなど）と名前で、画面の部品を探すメソッド。`screen.getByRole("heading", { name: "ログイン" })` は「『ログイン』という見出し」。見つからないとテストは失敗します。
- `.toBeInTheDocument()`（Vitest・Testing Library）：`expect(要素).toBeInTheDocument()` は「その要素が画面にある」ことを確かめます（jest-dom の機能）。
- `.getByLabelText()`（Vitest・Testing Library）：`screen.getByLabelText("従業員番号")` は、その名前の見出し（ラベル）が付いた入力欄を探します。見つからないとテストは失敗します。
- `<MemoryRouter>`（Vitest・Testing Library）：テストで、画面の部品を `<MemoryRouter>…</MemoryRouter>` で囲んで描きます。囲まないと、React Router の道具を使う部品はエラーになります。
- `it()`（Vitest・Testing Library）：`it("テストの名前", () => { … })` は、テストを 1 つ書きます。名前は日本語でもかまいません。
- `render()`（Vitest・Testing Library）：`render(<部品 />)` は、テストの中のにせのブラウザー（jsdom）に、部品を描きます。
- `expect()`（Vitest・Testing Library）：`expect(値)` は、「こうなっているはず」を確かめる書き方です。後ろに `.toBe(5)`・`.toBeInTheDocument()` などを付けます。違っていれば、その場でテストは失敗します。

1行ずつ。

- ① `it("ログイン見出しと従業員番号入力欄が表示されること", () => {` `it`（イット）は **1つのテスト** を定義します。第1引数は **テスト名**。日本語で「〜であること」と書くと、結果一覧がそのまま「仕様の一覧」になって読みやすいです。第2引数 `() => { ... }` がテストの中身。
- ② `render(<MemoryRouter><Login /></MemoryRouter>)` `render`（レンダー）で、`Login` 画面を **テスト用の仮想画面に描き** ます。`MemoryRouter` で包むのは§4.1③で説明したとおり、ルーターが必要だからです。これで「画面が表示された状態」が用意できました（準備＋実行を兼ねます）。
- ③ `expect(screen.getByRole("heading", { name: "ログイン" })).toBeInTheDocument();` 検証その1。日本語にすると「`heading`（見出し）**の役割を持ち、名前が "ログイン" の要素が、画面に存在することを期待する**」。
- `screen.getByRole(...)` … 画面から「役割（role）」で要素を探す。`"heading"` は `<h1>` 〜 `<h6>` などの見出し。
- `{ name: "ログイン" }` … その見出しの文字が「ログイン」であること。
- `.toBeInTheDocument()` …「画面（ドキュメント）の中に存在する」という検証（jest-domの機能）。
- `Login.tsx` には `<h2>ログイン</h2>` があるので、これが見つかります（`03` 章§3④で見たJSX）。

> **用語: ロール（role＝役割）** HTML要素が持つ「意味上の役割」。`<button>` は `button` 、`<h1>〜<h6>` は `heading` 、`<input>` は `textbox` など。RTL は「ユーザー（や読み上げソフト）から見た役割」で要素を探すことを推奨します。これが §2 で説明した「ユーザー視点のテスト」です。

- ④ `expect(screen.getByLabelText("従業員番号")).toBeInTheDocument();` 検証その2。「**ラベルが "従業員番号" の入力欄が、画面に存在することを期待する**」。
- `screen.getByLabelText("従業員番号")` … `<label>従業員番号</label>` と結びついた入力欄を探す。
- `Login.tsx` には `<label htmlFor="numberInput">従業員番号</label>` と `<input id="numberInput">` があり、`htmlFor` と `id` で結ばれています（ `03` 章§3⑥⑦）。だからこのラベル名で入力欄を取れます。

> **なぜラベルで探す？** 「ラベルと入力欄が正しく結びついているか」も同時に確認できるからです。これが結びついていないと、目の不自由な方の読み上げソフトが「何の入力欄か」を説明できません。ラベルで取れる＝アクセシビリティもOK、という一石二鳥のチェックです（ `03` 章§15）。

4.8 テスト2：ログイン成功時に `/` へ遷移すること（49〜69行目）

ここから「操作して、結果を待って、検証する」非同期テストが始まります。

> **▼ このコードがやること:** 「API が成功を返す」状況を仕込んでから、従業員番号を入力してログインボタンを押し、トップページ `/` へ遷移しようとしたかを確認します。ログイン成功の正常系テストです。

コピー

```
// ① 非同期テスト（async）
it("ログイン成功時に / へ遷移すること", async () => {
  // ② 【準備】次のpostは成功を返す
  mockedApi.post.mockResolvedValueOnce({
    //    サーバーが返すデータ（成功）
    data: { status: "success" },
  });

  // ③ 【準備】画面を描く
  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  // ④ 入力欄を取得
  const input = screen.getByLabelText("従業員番号");
  // ⑤ 【実行】"12345" を入力
  fireEvent.change(input, { target: { value: "12345" } });

  // ⑥ ボタンを取得
  const button = screen.getByRole("button", { name: "ログイン" });
  // ⑦ 【実行】ボタンをクリック
  fireEvent.click(button);

  // ⑧ 【検証】条件が満たされるまで待つ
  await waitFor(() => {
    //    "/" へ遷移しようとしたはず
    expect(mockedNavigate).toHaveBeenCalledWith("/");
  });
});
```

▶ ここで初めて出てくるコード

- `.mockResolvedValueOnce()`（Vitest・Testing Library）：にせの関数が、次の 1 回だけ「成功して、この値を返す」ようにします。`mockedApi.post.mockResolvedValueOnce({ data: … })` で、サーバーが成功の返事をした場面を作れます。
- `.change()`（Vitest・Testing Library）：`fireEvent.change(入力欄, { target: { value: "12345" } })` は、入力欄に文字を入れた時の動きを起こします。
- `.click()`（Vitest・Testing Library）：`fireEvent.click(ボタン)` は、テストの中で、ボタンを押した時の動きを起こします。
- `.toHaveBeenCalledWith()`（Vitest・Testing Library）：`expect(にせの関数).toHaveBeenCalledWith("/")` は「その関数が、この値を渡されて呼ばれた」ことを確かめます。
- `waitFor()`（Vitest・Testing Library）：`await waitFor(() => { expect(…) })` は、中の確かめが通るまで、少しずつ待ちながらくり返します。通信の後に画面が変わるのを待つ時に使います。

1行ずつ。

- ① `it("ログイン成功時に / へ遷移すること", async () => {` テスト名のあと、関数に `async`（エイシンク）が付いています。これは「**この中で `await`（待つ）を使う非同期テスト**」という印です（`04` 章の非同期処理）。API通信や遷移は「少し時間がかかる」ため、待つ必要があります。
- ② `mockedApi.post.mockResolvedValueOnce({ data: { status: "success" } });` **準備の山場。**にせの `post` に「**次に呼ばれたら1回だけ、成功してこのデータを返せ**」と仕込みます。
- `mockResolvedValueOnce(...)` …「Resolve（解決＝成功）する値を、Once（1回だけ）設定」。Promiseが成功する、つまり `await api.post(...)` が成功して中身を返す状況を演じます。
- `{ data: { status: "success" } }` … サーバーが返すであろうデータの形。`Login.tsx` は `response.data.status` が `"success"` なら「ログイン成功」とみなします。

なぜ `Once`（1回だけ）？ テストごとに「成功」「失敗」など仕込みを変えたいからです。`Once` を使えば、このテストの中の1回の `post` だけが成功を返し、他のテストには影響しません。`beforeEach` のリセット（§4.6）と合わせて、各テストの独立性を守ります。

- ③ `render(...)` … 画面を描く（§4.7②と同じ）。
- ④ `const input = screen.getByLabelText("従業員番号");` 従業員番号の入力欄を取得し、`input` という変数に入れます。次の操作で使い回すための取得です。
- ⑤ `fireEvent.change(input, { target: { value: "12345" } });` **実行その1。**入力欄に「12345」と打ち込む操作を発火します。
- `fireEvent.change(...)` …「入力内容が変化した（change）」というイベントを起こす。

- `{ target: { value: "12345" } }` …「変化先の値が "12345"」。ユーザーがキーボードで「12345」と打ったのと同じ状態になります。これで `Login.tsx` の `onChange` が呼ばれ、内部の state（従業員番号）が更新されます（`05` 章）。
- ⑥ `const button = screen.getByRole("button", { name: "ログイン" });`「ログイン」という名前のボタンを取得。`Login.tsx` の `<button type="submit">ログイン</button>` が見つかります（`03` 章 §3⑩）。
- ⑦ `fireEvent.click(button);` **実行その2。**ボタンをクリックする操作を発火。これでフォームが送信され、`handleSubmit` が動き、内部で `api.post(...)`（＝にせの post、成功を返す）が呼ばれます。
- ⑧ `await waitFor(() => { expect(mockedNavigate).toHaveBeenCalledWith("/"); });` **検証の山場。**
- `waitFor(() => { ... })` … 中の検証が **成功するまで、少しずつ何度も試しながら待つ** 道具。API通信→成功判定→遷移、という流れには **ごくわずかな時間** がかかります。クリック直後にすぐ検証すると「まだ遷移していない」と早とちりして失敗するので、`waitFor` で「遷移するまで待ってから」確認します。
- `expect(mockedNavigate).toHaveBeenCalledWith("/")` …「にせの遷移関数が、`"/"`（トップページ）という引数で **呼ばれたことを期待する**」。`toHaveBeenCalledWith(値)` は「その値で呼ばれたか」を確認するモック専用の検証です。
- `Login.tsx` は成功時に `navigate("/")` を呼ぶので、これが記録されているはず、という検証です。

> **用語: 非同期（あさんき／asynchronous）**「結果がすぐには返らず、少し待つ必要がある」処理のこと。通信・遷移などが該当します。だから `async` / `await` / `waitFor` でていねいに「待つ」のです。`04` 章の復習でもあります。

4.9 テスト3：ログイン失敗時にエラーメッセージが表示されること（71〜91行目）

> **▼ このコードがやること:**「API は成功で応答するが、中身が `status: "error"`（従業員が見つからない）」という状況を仕込み、ログインを試みて、**画面にエラーメッセージが表示される** ことを確認します。異常系（業務的な失敗）のテストです。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
it("ログイン失敗時にエラーメッセージが表示されること", async () => {
  // ① 【準備】postは成功で応答するが…
  mockedApi.post.mockResolvedValueOnce({
    //    中身は「error」＋メッセージ
    data: { status: "error", message: "従業員が見つかりません" },
  });

  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  const input = screen.getByLabelText("従業員番号");
  // ② 存在しない番号を入力
  fireEvent.change(input, { target: { value: "99999" } });

  const button = screen.getByRole("button", { name: "ログイン" });
  // ③ ボタンクリック
  fireEvent.click(button);

  // ④ 【検証】alert要素が現れるまで待ち
  expect(await screen.findByRole("alert")).toHaveTextContent(
    //    その中身がこの文言であること
    "従業員が見つかりません"
  );
});
```

▶ ここで初めて出てくるコード

- **`.findByRole()`**（Vitest・Testing Library）：`await screen.findByRole("alert")` は、その役割（role）の要素が出てくるまで少し待って探します。通信の後に出るメッセージなどを探す時に使います。
- **`.toHaveTextContent()`**（Vitest・Testing Library）：`expect(要素).toHaveTextContent("文字")` は「その要素の中に、その文字がある」ことを確かめます。

ポイントだけ。

- ① **`mockResolvedValueOnce({ data: { status: "error", message: "従業員が見つかりません" } })`** ここが §4.8 と違う点です。通信自体は **成功**（`mockResolvedValueOnce`）ですが、**中身の `status` が `"error"`**。これは「サーバーは正常に応答したが、業務的には失敗（その従業員番号は存在しない）」という状況です。`Login.tsx` はこのとき `message` を画面のエラー表示に出します。
- ② **`fireEvent.change(input, { target: { value: "99999" } });`** 存在しない番号「99999」を入力（テストの筋書きに合わせた値）。

- ④ `expect(await screen.findByRole("alert")).toHaveTextContent("従業員が見つかりません");` **`findByRole` の登場** です。
- `screen.findByRole("alert")` …「`alert`（警告）の役割を持つ要素が **現れるまで待って** 取得する」。`find` 系は **非同期で待つ** 取得です（§5で詳説）。
- `await` … 現れるまで待ちます。エラー表示は「クリック→通信→判定→画面更新」の後にやっと出るので、待つ必要があります。
- `Login.tsx` のエラー表示は `<div role="alert">{errorMessage}</div>`（`03` 章§3⑨）。`role="alert"` を持つので `findByRole("alert")` で取れます。
- `.toHaveTextContent("従業員が見つかりません")` … その要素の中身が、この文言を **含む** ことを検証（jest-domの機能）。

> **なぜ `getByRole` ではなく `findByRole` ？** エラー表示は **最初は画面に無く**、ログイン失敗後に **後から現れます**。`getBy`（即取得）だと「いま無いからエラー」になってしまいます。`findBy` は「現れるまで待つ」ので、後から出る要素にはこちらを使います。この使い分けが §5 の核心です。

4.10 テスト4：401エラー時に `/login` へ遷移すること（93〜113行目）

> ▼ **このコードがやること:** 「API が 401（未認証）エラーで失敗する」状況を仕込み、ログインを試みて、ログインページ `/login` へ強制遷移しようとしたかを確認します。セッション切れ時の挙動を守るテストです。

コピー

```
it("401エラー時に /login へ遷移すること", async () => {
  // ① 【準備】postは「失敗（reject）」する
  mockedApi.post.mockRejectedValueOnce({
    //    通信エラー。HTTPステータスは401
    response: { status: 401 },
  });

  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  const input = screen.getByLabelText("従業員番号");
  fireEvent.change(input, { target: { value: "12345" } });

  const button = screen.getByRole("button", { name: "ログイン" });
  fireEvent.click(button);

  // ② 【検証】遷移を待って確認
  await waitFor(() => {
    //    "/login" へ遷移したはず
    expect(mockedNavigate).toHaveBeenCalledWith("/login");
  });
});
```

▶ ここで初めて出てくるコード

- `.mockRejectedValueOnce()`（Vitest・Testing Library）: にせの関数が、次の 1 回だけ「失敗して、このエラーを出す」ようにします。サーバーがエラーを返した場面を作れます。

- ① `mockedApi.post.mockRejectedValueOnce({ response: { status: 401 } });` `mockRejectedValueOnce`（**リジェクト＝拒否・失敗**） の登場。「次の post は **失敗（例外を投げる）** し、このエラーオブジェクトを投げる」と仕込みます。
- `{ response: { status: 401 } }` … 投げるエラーの中身。`response` があるので §4.4 の `isAxiosError` が `true` を返し、「通信エラー」と判定されます。`status: 401` は「**認証が必要・セッション切れ**」のHTTPステータス（`07` 章）。
- `Login.tsx` は「通信エラーで status が 401 なら、ログインページ `/login` へ飛ばす」処理を持っています（CLAUDE.md の「401はログインへ」のルール）。

用語: 401（Unauthorized） HTTPステータスコードの1つで「**認証されていない／セッションが切れた**」という意味。アプリではこのとき「もう一度ログインして」とログイン画面へ戻します。`07` 章で扱った通信エラーの代表です。

- ② `await waitFor(() => { expect(mockedNavigate).toHaveBeenCalledWith("/login"); });` 「にせの遷移関数が `"/login"` で呼ばれた」ことを、遷移が起きるまで待って検証します。§4.8⑧と同じ形で、行き先だけ `/login` です。

4.11 テスト5：401以外のAPIエラー時にメッセージ表示（115〜132行目）

最後のテスト。401 **以外** の通信エラー（例：500 サーバーエラー）のときの挙動です。

> ▼ **このコードがやること:** 「API が 500（サーバーエラー）で失敗し、エラーメッセージ付きで応答する」状況を仕込み、ログインを試みて、**そのエラーメッセージが画面に表示される** ことを確認します。401は遷移、それ以外はメッセージ表示、という分岐の後者を守ります。

コピー

```
it("401以外のAPIエラー時にレスポンスのエラーメッセージが表示されること", async () => {
  // ① 【準備】postは失敗する
  mockedApi.post.mockRejectedValueOnce({
    //  500＋エラーメッセージ
    response: { status: 500, data: { message: "サーバーエラー" } },
  });

  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );

  const input = screen.getByLabelText("従業員番号");
  // ② 入力
  fireEvent.change(input, { target: { value: "12345" } });
  // ③ 取得とクリックを1行で
  fireEvent.click(screen.getByRole("button", { name: "ログイン" }));

  // ④ メッセージ表示を待って検証
  expect(await screen.findByRole("alert")).toHaveTextContent("サーバーエラー");
});
```

- ① `mockRejectedValueOnce({ response: { status: 500, data: { message: "サーバーエラー" } } });` 通信エラーで失敗。`status: 500` は「**サーバー内部のエラー**」。`data.message` に「サーバーエラー」という文言が入っています。`Login.tsx` は「401 **以外** の通信エラーなら、レスポンスの `message` を画面に表示する」処理を持っています。

- ③ `fireEvent.click(screen.getByRole("button", { name: "ログイン" }));` これまでは「取得して変数に入れてからクリック」でしたが、ここでは **取得とクリックを1行に** まとめています。`getByRole(...)` の結果を、変数に入れず直接 `fireEvent.click(...)` に渡しているだけで、動作は同じです。

- ④ `expect(await screen.findByRole("alert")).toHaveTextContent("サーバーエラー");`
  §4.9④と同じく、`findByRole("alert")` でエラー表示が **現れるまで待ち**、その中身が「サーバーエラー」であることを検証します。

4.12 5つのテストを表でふりかえる

#	テスト名	準備（仕込み）	実行	検証
1	見出しと入力欄が表示	なし（描くだけ）	描画	見出し・入力欄が存在
2	成功で `/` へ遷移	`mockResolvedValueOnce`（status:success）	入力→クリック	`navigate("/")` が呼ばれた
3	失敗でエラー表示	`mockResolvedValueOnce`（status:error）	入力→クリック	`alert` に「従業員が見つかりません」
4	401 で `/login` へ	`mockRejectedValueOnce`（status:401）	入力→クリック	`navigate("/login")` が呼ばれた
5	401以外でメッセージ	`mockRejectedValueOnce`（status:500）	入力→クリック	`alert` に「サーバーエラー」

> **この5つで、ログイン画面の「あり得る分岐」をほぼ網羅** しています。「正常表示・成功・業務失敗・認証切れ・サーバー障害」——ユーザーが遭遇するすべての結末を、コマンド1つで一瞬に検証できます。これがテストの威力です。

---

5. getBy / findBy / queryBy の使い分け

§4で `getByRole` `getByLabelText` `findByRole` が出てきました。RTL の要素取得には **3つの系統** があり、使い分けが超重要です。

系統	待つ？	見つからないとき	主な用途
`getBy...`	待たない（即取得）	**すぐエラー（失敗）**	「最初から画面にあるはず」の要素
`findBy...`	**待つ**（非同期）	しばらく待って無ければエラー	「後から現れる」要素（API後に出るエラー等）
`queryBy...`	待たない	`null` **を返す**（エラーにしない）	「無いはず」を確認したいとき

5.1 図で覚える

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
getBy    …「今ここにある？」即チェック。無ければ即・赤エラー
findBy   …「現れるまで待つよ」少しずつ何度も探す。時間切れで赤エラー
queryBy …「あるか無いか教えて」無くても怒らず null を返す
```

5.2 本アプリでの実例での使い分け

- `screen.getByLabelText("従業員番号")`（§4.7④, 4.8④…） 入力欄は **画面を描いた瞬間から存在** します。だから「待たない」`getBy` でOK。
- `screen.getByRole("button", { name: "ログイン" })`（§4.8⑥…） ボタンも最初からあります。`getBy` でOK。
- `screen.findByRole("alert")`（§4.9④, 4.11④） エラー表示は **ログイン失敗の後にやっと現れます**。最初は無いので、`getBy` だと「無い！」と即失敗します。`findBy` で「現れるまで待つ」のが正解。

> **使い分けの早見フレーズ「最初からある → getBy」「後から出る → findBy」「無いことを確かめる → queryBy」。** この3つを覚えれば、RTLの要素取得で迷いません。
>
> ⚠️ **よくある失敗:** API後に出るエラーを `getByRole("alert")` で取ろうとして、毎回「要素が見つからない」で落ちる。これは `findByRole`（待つ）に変えれば直ります。逆に、最初からある要素を `findBy` で取っても動きますが、無駄に待つので `getBy` が適切です。

---

6. フロントのテスト設定ファイル

テストを動かすための設定が3つのファイルにあります。それぞれ実コードで確認します。

6.1 vite.config.ts（テストの全体設定・全11行）

> ▼ **このコードがやること:** Vitest に「テストをどう動かすか」を伝える設定です。React を扱えるようにし、ブラウザのふり（jsdom）をさせ、テスト前の共通準備ファイルを指定します。

コピー

```
// ① Vitestの設定を書く道具
import { defineConfig } from 'vitest/config';
// ② Reactを扱うためのプラグイン
import react from '@vitejs/plugin-react';

// ③ 設定をエクスポート
export default defineConfig({
  // ④ Reactプラグインを有効化（JSXを理解できる）
  plugins: [react()],
  // ⑤ テスト固有の設定ここから
  test: {
    // ⑥ describe/it/expect を import 無しで使えるように
    globals: true,
    // ⑦ ブラウザのふりをする環境（§2.1）
    environment: 'jsdom',
    // ⑧ 各テスト前に読み込む共通準備ファイル
    setupFiles: './src/setupTests.ts',
  },
});
```

▶ ここで初めて出てくるコード

- **`defineConfig()`**（JavaScript・TypeScript）：`defineConfig({ … })` は、Vite・Vitest の設定ファイル（このアプリでは `frontend/vite.config.ts` ）で、設定を書くための関数です。書き間違いを、エディターが教えてくれるようになります。
- **`react()`**（React）：`react()` は、Vite に、React（JSX）のファイルを扱えるようにする部品（プラグイン）です。

1行ずつ。

- ① **`import { defineConfig } from 'vitest/config';`** … 設定を型安全に書くための関数。
- ② **`import react from '@vitejs/plugin-react';`** … React/JSX をテストでも理解できるようにするプラグイン。
- ③ **`export default defineConfig({ ... })`** … この設定オブジェクトを「この設定ファイルの中身」として外に出す。
- ④ **`plugins: [react()]`** … 上の React プラグインを有効化。これがないと `<Login />` のようなJSXが読めません。
- ⑤ **`test: { ... }`** … ここから下が「テスト専用」の設定。

- ⑥ `globals: true` … `describe` / `it` / `expect` を、各ファイルで `import` しなくても使えるようにする設定。

> **補足:** `Login.test.tsx` は1行目で明示的に `import { vi, describe, it, ... } from "vitest"` しています。`globals: true` でも明示 import でも、どちらでも動きます。明示する方が「どこから来た道具か」が分かりやすいので、本アプリは明示しています。

- ⑦ `environment: 'jsdom'` … テストの実行環境を **jsdom** にする。これで「ブラウザの画面（DOM）」が用意され、`render` や `screen` が動きます（§2.1）。
- ⑧ `setupFiles: './src/setupTests.ts'` … 各テストの前に、この準備ファイルを読み込む。中身は次の §6.2。

6.2 setupTests.ts（全5行）

> **▼ このコードがやること:** すべてのテストで使う「画面チェック専用の検証（jest-dom）」を、まとめて1回だけ読み込みます。各テストファイルで毎回 import しなくて済むようにする共通準備です。

コピー

```
// jest-dom adds custom jest matchers for asserting on DOM nodes.   // jest-domはDOM検証用の道具を足す
// allows you to do things like:                                    // 例えば次のような検証ができる：
// expect(element).toHaveTextContent(/react/i)                      //   要素が特定の文字を含むか
// learn more: https://github.com/testing-library/jest-dom         // 詳しくは公式へ
// jest-domを読み込む（全テストで有効に）
import '@testing-library/jest-dom';
```

- `import '@testing-library/jest-dom';` `toBeInTheDocument()` `toHaveTextContent()` などの「画面チェック専用の検証」を有効にします。`vite.config.ts` の `setupFiles` でこのファイルを指定しているので、**全テストの前に自動で読み込まれ**、どのテストでもこれらの検証が使えます。

> **補足:** `Login.test.tsx` は4行目でも `import "@testing-library/jest-dom/vitest";` を書いています。これは「Vitest向けに確実に登録する」ための念押しです。`setupTests.ts` と二重に見えますが、確実性のための実務的なやり方です。

6.3 package.json のテスト関連（31〜38行目, 57〜68行目）

> **▼ このコードがやること:** `npm` でテストを動かすための「ショートカット名（スクリプト）」と、テストに必要なライブラリの一覧（依存）を定義しています。

コピー

```
"scripts": {
  "start": "react-scripts start",    // 開発サーバー起動
  "build": "react-scripts build",    // 本番ビルド
  "test": "vitest",                  // ① テストを「見張りモード」で起動（変更を検知して自動再実行）
  "test:ui": "vitest --ui",          // ② ブラウザUI付きでテスト
  "test:run": "vitest run",          // ③ テストを「1回だけ」流して終了（CIや確認用）
  "eject": "react-scripts eject"
},
```

- ① `"test": "vitest"` … `npm test` で Vitest を **watch（見張り）モード** で起動。ファイルを保存するたびに、関係するテストを自動で流し直します。開発中に便利。
- ② `"test:ui": "vitest --ui"` … ブラウザの見やすい画面でテスト結果を確認するモード。
- ③ `"test:run": "vitest run"` … テストを **1回だけ実行して終了**。「提出前に全部通るか確認する」「自動チェック（CI）で回す」用途。**保守で最もよく使うのはこれ**（§13）。

依存ライブラリ側（`devDependencies`）には、テストに必要なものが並びます。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
"devDependencies": {
  "@testing-library/jest-dom": "^6.9.1",      // 画面チェック専用の検証（§6.2）
  "@testing-library/react": "^16.3.1",        // RTL本体（render/screen/fireEvent…）
  "@vitejs/plugin-react": "^5.1.2",           // ReactをVitestで扱うプラグイン（§6.1）
  "@vitest/coverage-v8": "^4.1.2",            // カバレッジ（どれだけテストしたか）計測
  "jsdom": "^27.3.0",                          // ブラウザのふり（§2.1）
  "vitest": "^4.0.16",                         // テスト実行ツール本体
  ...
}
```

> **用語: devDependencies（開発時依存）**「**開発・テストのときだけ** 使い、本番の動作には不要」なライブラリの置き場。テスト道具はユーザーに届ける製品には含めないので、ここに入れます。07 章で見た `dependencies`（本番でも使う）との違いです。
>
> **用語: カバレッジ（coverage＝網羅率）** コードの何％がテストで実行されたかの割合。`@vitest/coverage-v8` で計測できます。高いほど「テストでチェックされた範囲が広い」目安になりますが、100%が目的化しないよう注意（テストは「数」ではなく「重要な振る舞いを守っているか」が大事）。

---

7. バックのテスト道具：Django TestCase と APIClient

ここからバックエンド（Django/Python）のテストに移ります。考え方（3ステップ・モック）は同じですが、道具が変わります。

> **Django TestCase（テストケース）とは？** Django が用意する **テスト用の土台クラス**。これを継承（`07` 章の「親クラスを引き継ぐ」）してテストを書きます。最大の特徴は、**各テストの前に「まっさらなテスト用データベース」を用意し、テスト後に元に戻す** こと。本物のデータを汚さず、各テストが独立して動きます。
>
> **APIClient（エーピーアイ・クライアント）とは？** Django REST Framework が用意する **にせのブラウザ**。`self.client.get('/api/...')` や `self.client.post('/api/...')` と書くと、サーバーを起動せずに **アプリの中で直接APIを呼べます**。フロントの `fireEvent`（操作を発火）に相当する、「APIを叩く道具」です。

7.1 フロントとバックの道具の対応表

やりたいこと	フロント（Vitest/RTL）	バック（Django）
テストのグループ	`describe(...)`	`class TestXxx(TestCase):`（クラス）
1つのテスト	`it("...", () => {})`	`def test_xxx(self):`（メソッド）
各テスト前の準備	`beforeEach(...)`	`def setUp(self):`
操作する/呼ぶ	`fireEvent` / `render`	`self.client.get/post/...`
検証する	`expect(x).toBe(...)`	`self.assertEqual(x, ...)`

> **覚え方**: Django のテストは「**クラスがグループ、`test_` で始まるメソッドが1テスト、`setUp` が準備、`assertXXX` が検証**」。この4点セットで読めます。

---

8. 実コード全解説：test_backend.py の土台（冒頭〜setUp）

本アプリのバックテスト `kosu/tests/test_backend.py`（全2963行）の **土台部分** を読みます。最初に「全テストが共通で使う準備」が定義されており、ここを押さえると残りが一気に読めます。

8.1 インポート部（1〜24行目）

> **▼ このコードがやること**: テストに必要な道具・テスト対象（モデル・シリアライザ・関数）・標準ライブラリを読み込みます。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
# ① Djangoのテスト土台
from django.test import TestCase
# ② にせのブラウザ（API呼び出し用）
from rest_framework.test import APIClient
# ③ HTTPステータスの定数（200,401…）
from rest_framework import status
# ④ テスト対象のモデル（DBの表）たち
from kosu.models import (
    member, Business_Time_graph, kosu_division, def_choice,
    team_member, administrator_data, inquiry_data, AsyncTask, History
)
# ⑤ シリアライザ（データ変換役）たち
from kosu.views.serializers import (
    MemberSerializer, KosuSerializer, DefSerializer, DefChoiceSerializer,
    InquirSerializer, AdministratorSerializer, TaskSerializer, HistorySerializer
)
# ⑥ 共通関数
from kosu.utils.main_utils import validate_employee_no_logic, get_all_model_names_in_myapp
# ⑦ 工数計算のユーティリティ関数たち
from kosu.utils.kosu_utils import (
    time_index, break_time_process, kosu_write, break_time_delete,
    break_time_write, detail_list_summarize, judgement_check, kosu_sort,
    break_get, parse_break_time, get_week_of_month, kosu_division_dictionary,
    get_def_library_data, create_kosu_basic
)
# ⑧ JSON（データ形式）を扱う標準ライブラリ
import json
# ⑨ OS機能（ファイルパス等）
import os
# ⑩ 一時ファイルを作る道具
import tempfile
# ⑪ 日時を扱う（dt_moduleという別名で）
import datetime as dt_module
# ⑫ 日付・期間
from datetime import date, timedelta
# ⑬ Python標準のモック道具
from unittest.mock import patch, MagicMock
```

▶ ここで初めて出てくるコード

- `TestCase`（Python のテスト）: Django のテストの土台のクラス。`class 名前(TestCase):` の中に `def test_〜(self):` を書きます。テストごとに、試し用のまっさらなデータベースが使われます。
- `APIClient`（Python のテスト）: Django REST framework のテスト用の道具。`self.client = APIClient()` で作り、`self.client.post("/api/login/", データ, format="json")` のように、本物のサーバーを動かさずに API を呼べます。
- `MagicMock`（Python のテスト）: 何を呼ばれても、それらしく返事をする、にせの物。`patch` と組み合わせて、メールの送信やファイルの保存などを、本当にはやらずに試します。

要点だけ。

- ① `from django.test import TestCase` … §7のテスト土台。これを継承してテストクラスを作ります。
- ② `from rest_framework.test import APIClient` … にせのブラウザ。`self.client` として使います。
- ③ `from rest_framework import status` … `status.HTTP_200_OK` のような **読みやすい定数**。テストでは `200` のように数値を直接書く箇所もありますが、定数を使うと意図が明確になります。
- ④ **モデル群** … `member`（社員）・`Business_Time_graph`（工数記録）など、DBの「表」を表すクラス（CLAUDE.mdの「Key Models」）。テストはこれらを作って・調べます。
- ⑤ **シリアライザ群** … モデルのデータを、画面に渡せる形（JSON）に変換する役（`06`章相当）。これ自体もテスト対象。
- ⑥⑦ **ユーティリティ関数群** … 工数計算や検証のための部品関数。`test_backend.py` 後半でこれらを直接テストします（休憩時間の計算など）。
- ⑬ `from unittest.mock import patch, MagicMock` … Python標準の **モック道具**。バックでも外部処理（ファイル保存など）をにせものに差し替えるときに使います（§3のバック版）。

8.2 共通ヘルパー：make_break_times（27〜35行目）

▼ **このコードがやること:** 社員を作るときに必須の「休憩時間」フィールドを、まとめて一気に用意するための補助関数です。社員1人につき休憩時間の項目が大量にあるため、毎回手で書くと大変なので、関数で生成します。

コピー

```
def make_break_times():
    # 必須の break_time 一式を作る補助関数
    """Helper to generate all required break_time fields."""
    # ① 空の辞書（キー:値の入れ物）を用意
    bt = {}
    # ② 1〜6 まで繰り返す（直＝シフトが6種）
    for i in range(1, 7):
        # ③ break_time1〜6 に基本休憩を設定
        bt[f'break_time{i}'] = '#10401130'
        # ④ 残業休憩1
        bt[f'break_time{i}_over1'] = '#15101520'
        # ⑤ 残業休憩2
        bt[f'break_time{i}_over2'] = '#20202110'
        # ⑥ 残業休憩3
        bt[f'break_time{i}_over3'] = '#01400150'
    # ⑦ 出来上がった辞書を返す
    return bt
```

1行ずつ。

- ① `bt = {}` … 空の **辞書**（dictionary）を用意。辞書は「キー → 値」の対応表（`04` 章相当）。
- ② `for i in range(1, 7):` … `i` を 1, 2, 3, 4, 5, 6 と変えながら繰り返す。`range(1, 7)` は「1以上7未満」＝1〜6。本アプリは勤務形態（直＝シフト）が6種類あるため、それぞれに休憩時間があります（`member` モデル）。
- ③〜⑥ … `f'break_time{i}'` の `f'...{i}...'` は **f文字列**。`i` の値を文字列に埋め込みます。`i=1` のとき `'break_time1'`、`i=2` のとき `'break_time2'` …と、キー名が自動生成されます。値の `'#10401130'` は「10:40〜11:30 が休憩」を表す本アプリ独自の符号（`#` ＋ 開始時刻 ＋ 終了時刻、`08` 章の工数ロジック相当）。
- ⑦ `return bt` … 完成した辞書（`break_time1` 〜 `break_time6` と各 `over1〜3`、合計24項目）を返す。

> **なぜヘルパー関数にする？** 社員を作るたびに24個の休憩項目を手で書くのは大変で、ミスも出ます。1つの関数にまとめておけば、`**make_break_times()` の1行で全項目を渡せます（`**` の意味は次の §8.3⑦で説明）。「**繰り返す準備は関数化する**」は、テストを読みやすく・壊れにくくする定石です。

8.3 共通土台：BaseTestCase の setUp（38〜60行目）

> **▼ このコードがやること:** ほとんどのテストが共通で使う「前提データ」をまとめて用意する土台クラスです。にせのブラウザ・管理者設定・工数バージョン・管理者社員・一般社員を、各テストの前に毎回作ります。

コピー

```
# ① TestCaseを継承した共通土台
class BaseTestCase(TestCase):
    # ② 各テストの前に毎回呼ばれる準備
    def setUp(self):
        # ③ にせのブラウザを用意
        self.client = APIClient()

        # Create admin settings
        # ④ 管理者設定を1件作成
        self.admin_settings = administrator_data.objects.create(menu_row='20')

        # Create kosu_division (needed for login)
        # ⑤ 工数バージョンを1件作成（ログインに必要）
        self.division = kosu_division.objects.create(
            kosu_name='TestVer', kosu_title_1='作業A'
        )

        # ⑥ 休憩時間一式を生成（§8.2）
        break_times = make_break_times()

        # ⑦ 管理者権限を持つ社員を作成
        self.member = member.objects.create(
            employee_no=12345, name='テスト太郎', shop='P',
            #    権限あり・管理者あり・休憩一式を展開
            authority=True, administrator=True, **break_times
        )

        # ⑧ 一般（権限なし）社員を作成
        self.normal_member = member.objects.create(
            employee_no=99999, name='一般ユーザー', shop='P',
            #    権限なし・管理者でない
            authority=False, administrator=False, **break_times
        )
```

▶ ここで初めて出てくるコード

- **`**辞書`** （Python）：`member.objects.create(…, **break_times)` のように、呼ぶ時に辞書の前に `**` を付けると、辞書の中身を、名前付きの引数としてばらして渡します（`break_time1='#10401130', break_time1_over1='#15101520', …` を全部書いたのと同じ）。

1行ずつ。

- ① `class BaseTestCase(TestCase):` `TestCase` を **継承** した「共通の親クラス」。これ自体はテストを持たず、**他のテストクラスがこれを継承して使う土台** です（`class TestModels(TestCase)` ではなく `class TestLoginLogout(BaseTestCase)` のように）。共通準備を1か所に集約する設計。

- ② `def setUp(self):` `setUp`（セットアップ）は、フロントの `beforeEach`（§4.6）に相当。**各テストメソッドの直前に毎回** 自動で呼ばれ、前提データを整えます。

  > **なぜ毎回？** TestCase は各テストの後にDBを巻き戻す（リセットする）ので、毎回ここで前提を作り直さないと、データが空のままテストが始まってしまうからです。各テストが「同じきれいな前提」から始まる保証になります。

- ③ `self.client = APIClient()` にせのブラウザを用意し、`self.client` に持たせます。以後 `self.client.get(...)` でAPIを叩けます。`self.` を付けると、同じテストクラスの他のメソッドから使い回せます。

- ④ `self.admin_settings = administrator_data.objects.create(menu_row='20')` 管理者設定（`administrator_data` モデル）を **1件作成**。`objects.create(...)` は「この内容でDBに1行作る」Djangoの書き方（`06` 章）。`menu_row='20'` はメニュー表示行数などの設定値。作った結果を `self.admin_settings` に持たせ、後のテストで使います。

- ⑤ `self.division = kosu_division.objects.create(kosu_name='TestVer', kosu_title_1='作業A')` 工数の定義バージョン（`kosu_division`）を1件作成。コメント `# needed for login` のとおり、ログイン処理がこのバージョン情報を参照するため、**ログインを通すための前提** として用意します。

- ⑥ `break_times = make_break_times()` §8.2 のヘルパーで、休憩時間24項目の辞書を作ります。

- ⑦ `self.member = member.objects.create(employee_no=12345, ..., authority=True, administrator=True, **break_times)` **管理者権限つきの社員** を作成。

- `employee_no=12345` … 従業員番号（フロントのテストでも使った番号と同じ）。
- `authority=True` …「一般の管理機能を使える権限あり」。
- `administrator=True` …「最上位の管理者である」。
- `**break_times` … **辞書を「キー=値」の引数群に展開** する書き方（`**` ＝アンパック）。`break_time1='#10401130', break_time2=..., ...` と24個分を一気に渡すのと同じになります。これが §8.2 でヘルパーを作った狙いです。

> **用語: `**`（ダブルアスタリスク／辞書アンパック）** 辞書を関数の **キーワード引数** にバラして渡す書き方。`{'a':1, 'b':2}` に `**` を付けると `a=1, b=2` として渡されます。大量の引数を1行でスッキリ渡せます。

- ⑧ `self.normal_member = member.objects.create(employee_no=99999, ..., authority=False, administrator=False, **break_times)` **権限のない一般社員** を作成。`authority=False`・`administrator=False` がポイント。

  > **なぜ2種類の社員を作る？**「管理者ならできる・一般ユーザーならできない（403）」という **権限のテスト** をするためです（§10）。両方を用意しておくことで、`self.member`（管理者）と `self.normal_member`（一般）を使い分けて検証できます。

8.4 ログイン補助メソッド（62〜75行目）

> **▼ このコードがやること:** テストの中で「ログインした状態」を作るための補助メソッドです。多くのAPIはログイン必須なので、これを呼んでセッションを確立してから本題のAPIを叩きます。

コピー

```
    # ① 指定番号でログインする補助
    def login(self, employee_no=12345):
        """Helper to set session via login endpoint."""
        # ② ログインAPIをPOSTで叩く
        response = self.client.post(
            #    ログインのURL
            '/api/login/',
            # ③ 従業員番号をJSON文字列にして送る
            json.dumps({'employee_no': employee_no}),
            # ④ 「JSONを送る」と宣言
            content_type='application/json'
        )
        # ⑤ 応答を返す
        return response

    # ⑥ 管理者(12345)でログインする近道
    def login_as_admin(self):
        return self.login(12345)

    # ⑦ 一般(99999)でログインする近道
    def login_as_normal(self):
        return self.login(99999)
```

> **▶ ここで初めて出てくるコード**
>
> - **`.dumps()`**（Python）：`json.dumps(値, ensure_ascii=False)` は、Python の値を JSON の文字列にする関数です（`ensure_ascii=False` で日本語をそのまま）。

1行ずつ。

- ① **`def login(self, employee_no=12345):`** ログインを行う補助メソッド。引数 `employee_no=12345` は**デフォルト値つき**。何も指定しなければ管理者番号 12345 でログインします。
- ② **`response = self.client.post('/api/login/', ..., content_type='application/json')`** にせのブラウザで、ログインAPI `/api/login/` を **POST** で叩きます（`07` 章のPOST：データを送る通信）。フロントの `Login.tsx` がやっているのと同じことを、テスト内で再現しています。

- ③ `json.dumps({'employee_no': employee_no})` `{'employee_no': 12345}` という辞書を、**JSON文字列** に変換します。`json.dumps`（ダンプス）は「Pythonの辞書 → JSON文字列」の変換関数。サーバーへはJSON形式の文字列で送るためです。
- ④ `content_type='application/json'` 「いま送るデータはJSON形式ですよ」とサーバーに宣言するヘッダ。これがないとサーバーが中身を正しく解釈できません。
- ⑤ `return response` … サーバーの応答を返します。呼び出し側で「ログインが成功したか（status 200か）」などを検査できます。
- ⑥⑦ `login_as_admin` / `login_as_normal` それぞれ「管理者でログイン」「一般でログイン」の **近道メソッド**。`self.login(12345)` / `self.login(99999)` を呼ぶだけ。テスト本文で `self.login_as_admin()` と書けば、意図（管理者として操作）が一目で分かります。

> **なぜログイン補助を用意する？** ほとんどのAPIは「ログイン済み」が前提（CLAUDE.mdの「Session-based authentication」）。各テストで毎回ログイン手順を書くと冗長なので、1メソッドにまとめます。
> `self.login_as_admin()` の1行で「ログイン状態」を作れるので、各テストは **本題の検証に集中** できます。

---

9. 実コード全解説：TestModels（モデルのテスト）

`test_backend.py` の最初の本格テスト群 `TestModels`（81〜154行目）を読みます。**モデル（DBの表）が正しく作れて、文字列表現が期待どおりか** を確かめる、基本中の基本のテストです。

9.1 TestModels クラスの開始（81行目）

コピー

```
# TestCaseを直接継承（BaseTestCaseではない）
class TestModels(TestCase):
```

- ここは `BaseTestCase` ではなく `TestCase` を **直接継承** しています。理由は「モデルを作れること自体」をテストするので、`BaseTestCase` の前提データ（社員など）が **不要・むしろ邪魔** だから。各テスト内で必要なデータを最小限だけ作ります。

9.2 社員モデルの作成と文字列表現（82〜89行目）

> **▼ このコードがやること:** 社員（member）を1件作り、(1) 文字列にしたとき名前が出ること、(2) 従業員番号が正しく保存されたこと、の2点を確認します。

コピー

```
# ① テスト名：社員の作成と文字列表現
def test_member_creation_and_str(self):
    # ② 休憩時間一式を生成
    bt = make_break_times()
    # ③ 【準備/実行】社員を1件作成
    m = member.objects.create(
        employee_no=11111, name='山田太郎', shop='P',
        authority=False, administrator=False, **bt
    )
    # ④ 【検証】文字列化すると名前になる
    self.assertEqual(str(m), '山田太郎')
    # ⑤ 【検証】従業員番号が保存されている
    self.assertEqual(m.employee_no, 11111)
```

▶ ここで初めて出てくるコード

- `.assertEqual()`（Django）：テストで、2 つの値が同じかを確かめるメソッド。`self.assertEqual(res.status_code, 403)` 。

1行ずつ。

- ① `def test_member_creation_and_str(self):` メソッド名が `test_` **で始まる** のが超重要。Django は「`test_` で始まるメソッドだけ」をテストとして自動実行します。フロントの `it("...")` に相当。名前で「何をテストするか」が分かるようにします。
- ② `bt = make_break_times()` … 休憩時間一式を用意（§8.2）。
- ③ `m = member.objects.create(employee_no=11111, name='山田太郎', ...)` 社員を1件 **作成**。`objects.create(...)` でDBに1行作り、その結果（モデルのインスタンス）を `m` に入れます。ここが「準備＋実行」。
- ④ `self.assertEqual(str(m), '山田太郎')` **検証その1**。`assertEqual(a, b)` は「`a` と `b` が **等しいことを期待** する」検証。等しくなければ失敗。
- `str(m)` … 社員モデルを **文字列に変換** したもの。`member` モデルには「文字列にすると名前を返す」設定（`__str__` メソッド）があり、結果は `'山田太郎'` のはず。

  > **なぜ文字列表現をテスト？** Django の管理画面やログで、データは `str(...)` で表示されます。ここが正しく名前を返さないと、管理画面で「member object (1)」のような無意味な表示になり、運用が困ります。地味ですが大事な確認です。

- ⑤ `self.assertEqual(m.employee_no, 11111)` **検証その2**。作った社員の `employee_no`（従業員番号）が、指定した `11111` になっていることを確認。「ちゃんと保存されたか」のチェックです。

9.3 工数記録モデルの文字列表現（91〜102行目）

> **▼ このコードがやること:** 工数記録（Business_Time_graph）を1件作り、文字列化したときの形式が「ID__日付:従業員番号」になることを確認します。

コピー

```
def test_business_time_graph_creation_and_str(self):
    bt = make_break_times()
    # ① 紐づける社員を先に作る
    m = member.objects.create(
        employee_no=11111, name='山田太郎', shop='P',
        authority=False, administrator=False, **bt
    )
    # ② 工数記録を作成
    kosu = Business_Time_graph.objects.create(
        #   社員m・就業日2026-01-01
        employee_no3=11111, name=m, work_day2=date(2026, 1, 1),
        tyoku2='1', over_time=0
    )
    # ③ 期待する文字列を組み立て
    expected = f'{kosu.id}__2026-01-01:11111'
    # ④ 【検証】文字列が期待どおりか
    self.assertEqual(str(kosu), expected)
```

> **▶ ここで初めて出てくるコード**
>
> - **`date()`**（Python）：日付を表す型。`date(2026, 11, 10)` のように作ります。

- ① **社員を先に作る** … 工数記録は社員に **紐づく**（外部キー `name=m`）。先に社員 `m` を作ってから、それを参照します。
- ② **`Business_Time_graph.objects.create(employee_no3=11111, name=m, work_day2=date(2026,1,1), ...)`** 工数記録を1件作成。`name=m`（先ほどの社員）、`work_day2=date(2026, 1, 1)`（就業日。`date(年,月,日)` で日付を作る）、`tyoku2='1'`（勤務形態1）、`over_time=0`（残業0）。
- ③ **`expected = f'{kosu.id}__2026-01-01:11111'`** **期待する文字列** を f文字列で組み立てます。`kosu.id` はDBが自動採番したID（1など）。形式は「ID__日付:従業員番号」。
- ④ **`self.assertEqual(str(kosu), expected)`** 実際の `str(kosu)` が `expected` と一致するか検証。一致すれば、モデルの `__str__` が仕様どおりだと確認できます。

9.4 さまざまなモデルの作成テスト（104〜130行目）

短いテストが続きます。共通の型は「**作る → 文字列化や属性を検証**」。代表を1つずつ短く。

コピー

```
def test_kosu_division_creation(self):
    # 工数定義を作成
    div = kosu_division.objects.create(kosu_name='Ver2', kosu_title_1='TaskA')
    # 文字列に 'Ver2' が含まれるか
    self.assertIn('Ver2', str(div))
```

▶ ここで初めて出てくるコード

- `.assertIn()`（Django）：テストで、値が入れ物の中にあるかを確かめるメソッド。`self.assertIn("title", res.json())` 。

- `self.assertIn('Ver2', str(div))` `assertIn(a, b)` は「`a` が `b` の中に含まれることを期待 する」検証。ここでは「`str(div)` の中に `'Ver2'`（バージョン名）が含まれるか」。`assertEqual`（完全一致）より緩く、「一部に含まれていればOK」を確認したいときに使います。

コピー

```
def test_def_choice_creation(self):
    # 選択肢を作成
    choice = def_choice.objects.create(def_symbol='A', def_select='溶接作業')
    # 記号＋名称の結合が文字列表現
    self.assertEqual(str(choice), 'A溶接作業')
```

- 文字列表現が「記号 + 名称」（`'A'` + `'溶接作業'` = `'A溶接作業'`）になることを `assertEqual` で確認。

コピー

```
def test_administrator_data_creation(self):
    # 管理者設定を作成
    ad = administrator_data.objects.create(menu_row='30')
    # 文字列に '設定' を含むか
    self.assertIn('設定', str(ad))
```

- 管理者設定の文字列に「設定」という語が含まれるか（`assertIn`）。

9.5 自動削除のテスト（132〜141行目）— ふるまいの検証

▼ **このコードがやること:** タスク記録（AsyncTask）を上限を超えて作ったとき、「古いものが自動で消えて、件数が上限以内に収まる」というルールが守られているかを確認します。単なる作成ではなく「**しくみのふるまい**」をテストする例です。

コピー

```
def test_async_task_max_records_auto_deletion(self):
    """Create 1001+ records and verify oldest are deleted."""
    # Create 1002 records
    # ① 1002件のタスクを作成
    for i in range(1002):
        AsyncTask.objects.create(
            #    task_00000, task_00001, ...
            task_id=f'task_{i:05d}', status='success'
        )
    # ② 件数は上限以内のはず
    self.assertLessEqual(AsyncTask.objects.count(), AsyncTask.MAX_RECORDS)
    # Oldest task should have been deleted
    # ③ 最古は消えているはず
    self.assertFalse(AsyncTask.objects.filter(task_id='task_00000').exists())
```

▶ ここで初めて出てくるコード

- **`.assertLessEqual()`**（Python のテスト）： `self.assertLessEqual(a, b)` は「a が b 以下」を確かめます。違えばテストは失敗します。
- **`.assertFalse()`**（Python のテスト）： `self.assertFalse(値)` は「値が偽（False）」を確かめます。
- **`{値:0幅}`**（Python）：f 文字列の `{42:05d}` の `05` は、「5 けたになるまで、前を 0 で埋める」指定です（ `00042` ）。
- **`{値:d}`**（Python）：f 文字列の `{n:d}` の `d` は、整数として表示する指定です。

- ① **`for i in range(1002):`** … 1002件のタスクを作成。`f'task_{i:05d}'` の `:05d` は「5桁・ゼロ埋めの整数」フォーマット。`i=0` なら `'task_00000'`、`i=1` なら `'task_00001'` …と連番のIDを作ります。
- ② **`self.assertLessEqual(AsyncTask.objects.count(), AsyncTask.MAX_RECORDS)`**
  `assertLessEqual(a, b)` は「`a ≤ b`（a が b 以下）を期待」。作った件数が、モデルが定める上限 `MAX_RECORDS` を **超えていない** ことを確認。`AsyncTask` には「上限を超えたら古いものを自動削除する」しくみがあり、それが効いているかを見ています。
- ③ **`self.assertFalse(AsyncTask.objects.filter(task_id='task_00000').exists())`**
  `assertFalse(x)` は「`x` が **偽**（False） であることを期待」。`...filter(task_id='task_00000').exists()` は「最古のタスクがまだ存在するか」。これが `False`（＝もう存在しない）なら、最古が自動削除されたと確認できます。

なぜこのテストが大事？ これは「データを作れる」ではなく「**上限超過時に古いものが消える、という設計どおりのふるまい**」を検証しています。もし将来この自動削除のコードを壊してしまっても、このテストが赤くなって即座に教えてくれます。「**仕様（こうあるべき）」をコードで固定する** のがテストの真価です。

9.6 履歴モデルの文字列表現（149〜154行目）

コピー

```
def test_history_save_works(self):
    # 履歴を1件作成
    h = History.objects.create(
        #   操作=CREATE, 対象表=member
        operation='CREATE', table_name='member',
        #   対象ID=1, 変更内容=辞書
        record_id=1, changes={'field': 'value'}
    )
    # 文字列が決まった形式になるか
    self.assertEqual(str(h), 'CREATE on member (ID: 1)')
```

- 履歴（History）モデルの文字列表現が `'CREATE on member (ID: 1)'` という決まった形式になるかを検証。`changes={'field': 'value'}` のように、変更内容を **辞書のまま** 保存できる点も確認できます（`06` 章の監査ログ）。

---

10. APIの 401 / 403 / 200 を確かめるテストの考え方

`test_backend.py` の中核は「**APIが正しい権限チェックをするか**」のテストです。ここがバックテストの最重要ポイントなので、考え方を整理します。

10.1 3つのステータスの意味

ステータス	意味	いつ返る
200 OK	成功	ログイン済み・権限ありで、正しくデータが返るとき
401 Unauthorized	未認証	**ログインしていない**（セッションが無い）とき
403 Forbidden	権限不足	ログイン済みだが、**その操作の権限が無い** とき

> **401と403の違い（重要）** - 401 =「**あなたが誰か分からない**」。ログインしていない状態。→ ログイン画面へ。 - 403 =「**あなたが誰かは分かるが、これをする権限が無い**」。ログイン済みの一般ユーザーが、管理者専用機能を触ったときなど。
>
> たとえ話：401は「**入館証を持たずにビルに入ろうとした**」（そもそも誰か不明）。403は「**入館証はあるが、社長室には入れない**」（身元は分かるが権限不足）。

10.2 実例：メニューAPIの3パターン（197〜229行目）

`TestMainMenuViews` から、この3パターンが揃った例を読みます。

▼ **このコードがやること:** メニューAPIに対して、(1) 未ログイン→401、(2) 管理者→200、(3) 権限ありメニューに一般ユーザー→403、の3つの状況を、それぞれ別テストで確認します。

コピー

```
# ① 未ログインのとき
def test_menu_get_without_login(self):
    #    そのままGET（ログインしない）
    response = self.client.get('/api/main_menu/')
    #    → 401が返るはず
    self.assertEqual(response.status_code, 401)
```

▶ **ここで初めて出てくるコード**

- `.status_code`（Django）: テストの答えの `res.status_code` は、HTTP の状態コード（`200`・`403` など）です。

- ① ログインせずにいきなりAPIを叩き、`response.status_code`（応答のステータスコード）が **401** であることを確認。「未ログインは弾く」を保証。

コピー

```
# ② ログイン済みのとき
def test_menu_get_with_login(self):
    #    管理者でログイン（§8.4）
    self.login_as_admin()
    #    GET
    response = self.client.get('/api/main_menu/')
    #    → 200（成功）
    self.assertEqual(response.status_code, 200)
    #    応答に 'login_data' が含まれるか
    self.assertIn('login_data', response.data)
    #    応答に 'admin_data' が含まれるか
    self.assertIn('admin_data', response.data)
```

- ② `self.login_as_admin()` で **先にログイン** してからGET。今度は **200**（成功）になり、さらに `response.data`（応答の中身）に `'login_data'` `'admin_data'` というキーが含まれることを `assertIn` で確認。「正しくデータを返すか」まで見ています。

コピー

```
# ③ 権限なしユーザーが権限要メニューへ
def test_member_menu_without_authority(self):
    #     一般ユーザーでログイン
    self.login_as_normal()
    #     権限が必要なメニューにGET
    response = self.client.get('/api/member_menu/')
    #     → 403（権限不足）
    self.assertEqual(response.status_code, 403)
```

- ③ **一般ユーザー** で（`login_as_normal`）、人員管理メニュー（管理者権限が要る）にアクセス。`authority=False` なので **403** が返るはず。「権限の無い人を弾く」を保証します。

> **この3点セットがAPIテストの王道** 1つのAPIに対して「**未ログイン→401**」「**正しい権限→200**」「**権限不足→403**」の3テストを書く。これで「認証と権限のガードが効いているか」が完全に守られます。`test_backend.py` 全体が、ほぼこの型の繰り返しでできています（メニュー・人員・工数・定義・問い合わせ・管理者…すべてに対して）。

10.3 実例：作成・更新・削除の検証（298〜381行目より）

権限だけでなく「データ操作が正しく行われたか」も検証します。`TestMemberViews` から。

コピー

```
# 社員を新規作成できること
def test_member_new_post_creates_member(self):
    # ① 管理者でログイン
    self.login_as_admin()
    # ② 送る社員データ
    data = {
        'employee_no': 55555, 'name': '新規太郎', 'shop': 'P',
        'authority': False, 'administrator': False,
    }
    # ③ 作成API（POST）
    response = self.client.post('/api/member_new/', data, format='json')
    # ④ 201（作成成功）
    self.assertEqual(response.status_code, 201)
    # ⑤ DBに実際に作られたか
    self.assertTrue(member.objects.filter(employee_no=55555).exists())
```

> **▶ ここで初めて出てくるコード**
>
> - `.assertTrue()`（Python のテスト）：`self.assertTrue(値)` は「値が真（True）」を確かめます。

- ③ `self.client.post('/api/member_new/', data, format='json')` 社員作成APIをPOSTで叩く。`format='json'` で「JSONとして送る」（§8.4③の `json.dumps` の簡単版。APIClientがやってくれます）。
- ④ `self.assertEqual(response.status_code, 201)` **201 Created**（新規作成成功のステータス）を確認。200と区別し、「新しく作られた」ことを表します。
- ⑤ `self.assertTrue(member.objects.filter(employee_no=55555).exists())` `assertTrue(x)` は「`x` が **真**（True） を期待」。「従業員番号55555の社員が **実際にDBに存在する**」ことを確認。APIが200を返すだけでなく、**本当にデータが作られたか** までチェックするのが堅実なテストです。

コピー

```
# 社員を削除できること
def test_member_delete(self):
    self.login_as_admin()
    # ① 削除API（DELETE）
    response = self.client.delete(
        f'/api/member_delete/{self.normal_member.employee_no}/'
    )
    # ② 204（削除成功・中身なし）
    self.assertEqual(response.status_code, 204)
    # ③ DBから消えたか
    self.assertFalse(member.objects.filter(employee_no=99999).exists())
```

- ① `self.client.delete(...)` … DELETEメソッドでAPIを叩く（`07` 章：削除の通信）。URLに削除対象の従業員番号を埋め込みます。
- ② `assertEqual(response.status_code, 204)` … **204 No Content**（削除成功・返す中身なし）を確認。
- ③ `assertFalse(...exists())` … 削除後、その社員がDBに **存在しない**（`False`）ことを確認。「ちゃんと消えたか」の検証です。

> **作成は201・削除は204・取得や更新は200。**HTTPステータスを正しく確認することで、APIの「契約」を守れます（`07` 章のステータスコード）。

---

11. assertXXX メソッド早見表

ここまでに出てきた検証メソッドを一覧にします。バックテスト（Django/unittest）の `self.assertXXX(...)` 系です。

メソッド	意味（〜を期待する）	例
`assertEqual(a, b)`	a と b が **等しい**	`assertEqual(str(m), '山田太郎')`
`assertNotEqual(a, b)`	a と b が **等しくない**	
`assertTrue(x)`	x が **真**（True）	`assertTrue(...exists())`
`assertFalse(x)`	x が **偽**（False）	`assertFalse(...exists())`
`assertIn(a, b)`	a が b に **含まれる**	`assertIn('login_data', response.data)`
`assertNotIn(a, b)`	a が b に **含まれない**	
`assertIsNone(x)`	x が **None**（無）	`assertIsNone(err)`
`assertIsNotNone(x)`	x が **None でない**	`assertIsNotNone(err)`
`assertLessEqual(a, b)`	a ≤ b（a が b 以下）	`assertLessEqual(count, MAX)`
`assertGreaterEqual(a, b)`	a ≥ b（a が b 以上）	`assertGreaterEqual(week, 1)`
`assertRaises(例外)`	中の処理が **その例外を投げる**	`with self.assertRaises(ValueError):`

`assertRaises` の使い方（1219行目あたりの例）

コピー

```
with self.assertRaises(ValueError):          # この中で ValueError が起きることを期待
    parse_break_time(None, '...', jst)        # None を渡すとエラーになるはず
```

「**わざと不正な入力を与え、ちゃんとエラーになるか**」を確認するテスト。`with` ブロックの中で指定の例外が起きれば成功、起きなければ「エラーになるべきなのにならなかった」で失敗します。**異常系（エラーになるべき場合）のテスト** に使います。

11.1 フロントの expect 系との対応

フロント（Vitest/jest-dom）	バック（Django）	意味
`expect(a).toBe(b)`	`assertEqual(a, b)`	等しい
`expect(x).toBeTruthy()`	`assertTrue(x)`	真
`expect(x).toBeFalsy()`	`assertFalse(x)`	偽
`expect(el).toBeInTheDocument()`	（DOM特有・フロントのみ）	画面に存在
`expect(el).toHaveTextContent(s)`	`assertIn(s, ...)`	文字を含む
`expect(fn).toHaveBeenCalledWith(x)`	（モック特有）	x で呼ばれた

> **本質は同じ:** フロントもバックも「**期待を書いて、違えば失敗**」というアサーションの考え方は完全に共通です。書き方（ `expect(...).toBe(...)` か `self.assertEqual(...)` か）が違うだけです。

---

12. バックのテスト設定（test_settings.py）

バックのテストは、本番とは **別の設定** で動かします。`hozen_another/test_settings.py`（全31行）を読みます。

> **▼ このコードがやること:** テスト専用の設定です。本番設定を引き継ぎつつ、(1) DBをメモリ上の使い捨てSQLiteに、(2) 非同期処理を即時実行に、(3) マイグレーションを省略、に切り替えます。テストを速く・安全に・独立して動かすための工夫です。

コピー

```
# ① 本番設定をすべて引き継ぐ
from .settings import *

# ② DBの設定を上書き
DATABASES = {
    'default': {
        #    SQLiteを使う（PostgreSQLでなく）
        'ENGINE': 'django.db.backends.sqlite3',
        #    メモリ上のDB（ファイルに残さない）
        'NAME': ':memory:',
    }
}

# Disable signals during tests to avoid side effects
# ③ 「今はテスト中」フラグ（副作用を抑える目印）
TESTING = True

# Disable django-q cluster requirement for tests
# ④ 非同期タスクの設定
Q_CLUSTER = {
    'name': 'test',
    #    'sync': True ＝ 非同期処理を「その場で即実行」
    'sync': True,
    'timeout': 60,
    'orm': 'default',
}

# Custom test runner that creates tables from models directly (bypasses migrations)
# ⑤ マイグレーションを無効化する仕掛け
class DisableMigrations:
    """Fake migration module that tells Django to skip migrations and use syncdb."""
    def __contains__(self, item):
        return True
    def __getitem__(self, item):
        return None

# ⑥ マイグレーションを飛ばし、モデルから直接表を作る
MIGRATION_MODULES = DisableMigrations()
```

▶ ここで初めて出てくるコード

- **`__contains__`**（Python）：`def __contains__(self, item):` を書いたクラスの物は、`item in 物` と書けるようになります。
- **`__getitem__`**（Python）：`def __getitem__(self, item):` を書いたクラスの物は、`物[item]` と書いて中身を取り出せるようになります。

要点を1つずつ。

- ① `from .settings import *` 本番の設定（`settings.py`）を **まるごと取り込み**、その上で必要な箇所だけ上書きします。「本番とほぼ同じ環境で、テストに都合の悪い部分だけ変える」やり方。
- ② **DATABASES を `sqlite3` の `:memory:` に上書き** 本番は PostgreSQL（CLAUDE.md）ですが、テストでは **SQLite のメモリDB** を使います。

  > **なぜ？** メモリ上のDBは **超高速** で、テストが終われば **跡形もなく消える**。本物のDBを汚す心配がなく、毎回まっさらから始められます。テストの「速さ」と「安全（独立性）」の両立です。

- ③ `TESTING = True` 「いまテスト中」という目印。本体コードはこのフラグを見て、テスト中は不要な副作用（メール送信や外部通知など）を抑制できます。コメントの「Disable signals」が示すとおり、シグナル（モデル変更時の自動処理。CLAUDE.mdの `signals.py`）の暴発を防ぐためにも使われます。
- ④ `Q_CLUSTER` の `'sync': True` 本アプリは Django-Q で **非同期タスク**（バックアップ等）を裏で動かします（CLAUDE.md）。テストでは `'sync': True` にして、非同期タスクを **その場で即時実行** させます。

  > **なぜ？** テストで「裏で後から実行される」と、結果を待てず検証できません。`sync: True` で「呼んだら即その場で実行」にすると、結果をすぐ検証できます。フロントの「モック」と同じ発想（テスト都合に合わせる）です。

- ⑤⑥ `DisableMigrations` **でマイグレーションを飛ばす** 通常、DBの表はマイグレーション（`06` 章の `makemigrations` / `migrate`）で作ります。テストでは、それを飛ばして **モデル定義から直接、表を作ります**。

  > **なぜ？** コメントにあるとおり、PostgreSQL では許される書き方（`max_length` 無しの文字列フィールド等）が SQLite では問題になることがあるため、マイグレーションを経由せず「モデルから直接」表を作って回避しています。これにより、メモリ上の SQLite でもテストが安定して動きます。

> **要するに test_settings.py は:** 「本番と似た設定のまま、**DBを使い捨て・処理を即時・準備を最速** に切り替えて、テストを高速・安全・独立に回すための専用設定」です。だから実行時に `--settings=hozen_another.test_settings` で **この設定を指定** します（次の §13）。

---

13. テストの実行コマンドと出力例

道具と中身が分かったので、実際に **流す** 方法です。

13.1 フロントのテストを流す

`frontend/` ディレクトリで実行します。

コピー

```
# テストを1回流して終了（提出前の確認はこれ）
npm run test:run
# 見張りモード（保存のたび自動再実行。開発中はこれ）
npm test
# ブラウザUIで結果を見る
npm run test:ui
```

▼ こう表示されれば成功（ `npm run test:run` ）：

コピー

```
✓ src/tests/MainPage/Login.test.tsx (5 tests) 123ms
  ✓ Login Component > ログイン見出しと従業員番号入力欄が表示されること
  ✓ Login Component > ログイン成功時に / へ遷移すること
  ✓ Login Component > ログイン失敗時にエラーメッセージが表示されること
  ✓ Login Component > 401エラー時に /login へ遷移すること
  ✓ Login Component > 401以外のAPIエラー時にレスポンスのエラーメッセージが表示されること

Test Files  1 passed (1)
     Tests  5 passed (5)
```

緑色の `✓` （チェックマーク）と `passed` （合格）が並べば成功です。`describe` の名前（Login Component）と `it` の名前（各テスト）が、そのまま見出しになっているのが分かります。

13.2 バックのテストを流す

プロジェクトのルートディレクトリ（`manage.py` のある場所）で実行します。

コピー

```
python manage.py test --settings=hozen_another.test_settings
```

▶ ここで初めて出てくるコード

- **`python --settings`** （Django の道具）：`python manage.py test --settings=hozen_another.test_settings` の `--settings` は「この設定ファイルを使う」という指定です。テストの時だけ、試し用のデータベースなどの設定に切り替えます。

- `python manage.py test` … Django のテストを実行するコマンド。
- `--settings=hozen_another.test_settings` … **§12 のテスト専用設定を使う** 指定。これを忘れると本番DB設定で動こうとして失敗します。**この指定は必須** と覚えてください。

特定のクラスやテストだけ流すこともできます（速くて便利）。

コピー

```
# TestModels クラスだけ流す
python manage.py test kosu.tests.test_backend.TestModels --settings=hozen_another.test_settings

# 1つのテストメソッドだけ流す
python manage.py test kosu.tests.test_backend.TestModels.test_member_creation_and_str --settings=hozen_another.test_settings
```

▼ こう表示されれば成功:

コピー

```
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....................................................
----------------------------------------------------------------------
Ran 120 tests in 4.231s

OK
Destroying test database for alias 'default'...
```

- `.`（ドット）1つが **テスト1件の成功**。
- 最後の `OK` が「**全部合格**」の合図。
- `Creating test database` / `Destroying test database` … §12で見たとおり、テスト用DBを作って・壊しています。

⚠️ **失敗があると、`OK` の代わりに `FAILED (failures=1)` のように出て、`.` の代わりに `F`（失敗）や `E`（エラー）が表示されます。**その読み方は次の §14。

---

14. テストが失敗したときの読み方と対応

テストが赤くなったら、慌てず **メッセージを読む** のが鉄則です。テストは「どこが・どう期待と違ったか」を必ず教えてくれます。

14.1 バックの失敗メッセージの読み方

※説明用の簡易例（実際の失敗出力の典型）

コピー

```
======================================================================
FAIL: test_member_creation_and_str (kosu.tests.test_backend.TestModels)
----------------------------------------------------------------------
Traceback (most recent call last):
  File ".../test_backend.py", line 88, in test_member_creation_and_str
    self.assertEqual(str(m), '山田太郎')
AssertionError: 'member object (1)' != '山田太郎'
----------------------------------------------------------------------
```

読み解き方：

1. `FAIL: test_member_creation_and_str` … どのテストが落ちたか（名前で分かる）。
2. `line 88, in ...` … どの行で落ちたか。
3. `self.assertEqual(str(m), '山田太郎')` … どの検証で落ちたか。
4. `AssertionError: 'member object (1)' != '山田太郎'` … **期待と実際の違い**。「実際は `'member object (1)'` だったが、`'山田太郎'` を期待していた（`!=` ＝等しくない）」。

→ ここから「`member` モデルの `__str__`（文字列表現）が壊れている（名前を返していない）」と推理できます。**実際の値が左、期待が右** に並ぶことが多いので、その差分を見ます。

14.2 フロントの失敗メッセージの読み方

> ※説明用の簡易例

コピー

```
 FAIL  src/tests/MainPage/Login.test.tsx > Login Component > ログイン成功時に / へ遷移すること
AssertionError: expected "mockedNavigate" to be called with arguments: [ '/' ]

Number of calls: 0
```

- どのテストが落ちたか（`>` でグループ＞テスト名）。
- `expected ... to be called with [ '/' ]` …「`/` で呼ばれることを期待した」。
- `Number of calls: 0` …「実際は **0回** 呼ばれた」＝遷移していない。

→「クリックしても遷移していない」。原因候補は「モックの仕込み忘れ」「`waitFor` で待たずに即検証して早すぎた」「本体の成功判定の条件が変わった」など。

14.3 失敗時の対応フロー

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
1. メッセージを読む          … どのテスト・どの行・期待と実際の差は？
2. 「テストが正しい」か       … 仕様変更でテストの方を直すべき場合もある
   「本体が壊れた」か を判断
3a. 本体が壊れた             → 本体コードを直す（テストは正しい仕様の証）
3b. 仕様が変わった           → テストを新仕様に合わせて更新する
4. 再実行して緑になるまで    … 1つ直すごとに流し直す
```

> ⚠️ **やってはいけないこと:** 失敗したテストを **理解せずに消す・コメントアウトする**。それは「健康診断で異常が出たから検査項目を削った」のと同じで、本当の不具合を見逃します。**まず原因を理解** してから、本体を直すかテストを直すかを判断します。
>
> **なぜテストの方を直すこともある？** 仕様が正式に変わったとき（例：成功時の遷移先が `/` から `/home` に変更）、古いテストは「`/` を期待」のままなので落ちます。この場合はテストを新仕様（`/home`）に更新するのが正解。**テストは「現在の正しい仕様」を映す鏡** なので、仕様が変われば一緒に更新します。

---

15. 新しいテストの足し方

機能を追加・修正したら、テストも足すのが理想です。**既存のテストをコピーして一部を変える** のが最速・最安全です。

15.1 フロントに1つ足す（型をなぞる）

`Login.test.tsx` のテスト2（§4.8）をコピーし、「番号が空のときボタンを押してもAPIを呼ばないこと」を確かめる例を作るなら、こう考えます。

> ※説明用の簡易例（足し方のイメージ）

コピー

```
// ① 新しい仕様を名前に
it("従業員番号が空ならログインAPIを呼ばないこと", () => {
  // ② 準備：描く
  render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  );
  // ③ 実行：空のままクリック
  fireEvent.click(screen.getByRole("button", { name: "ログイン" }));
  // ④ 検証：postは呼ばれていないはず
  expect(mockedApi.post).not.toHaveBeenCalled();
});
```

▶ ここで初めて出てくるコード

- `.toHaveBeenCalled()`（Vitest・Testing Library）：`expect(にせの関数).toHaveBeenCalled()` は「1 回以上呼ばれた」ことを確かめます。`.not.toHaveBeenCalled()` なら「1 回も呼ばれていない」。

- 既存テストの「`render` → `fireEvent` → `expect`」の骨格をそのまま流用し、**仕込みと検証だけ** を目的に合わせて変えます。
- `expect(...).not.toHaveBeenCalled()` … `not` を挟むと「**呼ばれていないことを期待**」になります。

注: これは「足し方の型」を示す説明用の例です。実際に追加する際は、`Login.tsx` の本当の挙動（空欄時に `required` でブラウザが送信を止めるか等）を確認して、テストが実態と合うようにします。

15.2 バックに1つ足す（型をなぞる）

`test_backend.py` の §10.2 のパターンをコピーし、新しいAPIに「未ログイン→401」を足すなら：

※説明用の簡易例

コピー

```
# ① 新APIの未ログインテスト
def test_new_api_without_login(self):
    # ② ログインせずGET
    response = self.client.get('/api/new_endpoint/')
    # ③ 401を期待
    self.assertEqual(response.status_code, 401)
```

- 既存の `test_menu_get_without_login`（§10.2①）と **ほぼ同じ形**。URLだけ新しいAPIに変えます。

- `BaseTestCase` を継承したクラスに置けば、`self.client` や `self.login_as_admin()` がそのまま使えます。

15.3 テストを足す手順まとめ

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
1. 似た既存テストを見つける          … 同じ種類（表示/成功/権限…）を探す
2. それをコピーして名前を変える       … it("...") / def test_xxx の名前
3. 準備（仕込み）を目的に合わせる     … mock... / data = {...}
4. 実行を目的に合わせる              … 操作する要素・叩くURL
5. 検証を目的に合わせる              … expect / assert の期待値
6. 流して緑になるか確認              … npm run test:run / python manage.py test ...
```

> **黄金ルール:「バグを直したら、そのバグを再現するテストを1つ足す」**。これで「同じバグが二度と戻ってこない（リグレッションしない）」ことが保証されます。本アプリの `test_backend.py` が大量のテストを持つのは、こうして1つずつ「守るべき仕様」を積み上げてきた結果です。

16. トラブルシューティング

症状	原因	対処
`python manage.py test` がDB接続で失敗する	`--settings=hozen_another.test_settings` を付け忘れ	必ずこの設定を付ける（§13.2）
フロントで `Cannot find ... TitleRogo.png`	画像のモック漏れ	`vi.mock("...png", () => ({ default: "..." }))`（§4.2）を確認
`useNavigate() may be used only in the context of a Router`	ルーターで包んでいない	`<MemoryRouter>` で `<Login />` を包む（§4.1③）
エラー表示の検証が毎回落ちる	`getByRole("alert")`（待たない）を使った	`findByRole("alert")`（待つ）に変える（§5）
`toHaveBeenCalledWith` が `Number of calls: 0`	`waitFor` で待たず即検証／成功仕込み漏れ	`await waitFor(...)` で待つ・`mockResolvedValueOnce` を確認（§4.8）
前のテストの影響で結果が変わる	モックのリセット漏れ	`beforeEach(vi.clearAllMocks)` / Django は `setUp` でDBリセット（§4.6,§8.3）
`toBeInTheDocument` is not a function	jest-dom 未読込	`setupTests.ts` の import と `setupFiles` 設定を確認（§6.2）
403 が来てほしいのに 200 が来る	一般ユーザーでなく管理者でログインしている	`login_as_normal()` を使う（§10）
201/204 を期待したのに 400	送ったデータの不備（重複番号・必須欠け）	`data` の中身を見直す（§10.3）
`AssertionError: ... != ...`	期待値と実際値の食い違い	左（実際）と右（期待）の差を読み、本体かテストを直す（§14）

17. 演習問題

手を動かすと定着します。**ローカル環境**（壊しても安全）で試してください。

1. **フロントを流す**：`frontend/` で `npm run test:run` を実行し、5件すべてが緑（passed）になることを確認する。
2. **バックを流す**：ルートで `python manage.py test kosu.tests.test_backend.TestModels --settings=hozen_another.test_settings` を実行し、`OK` が出ることを確認する。

3. **わざと壊して観察**：`Login.test.tsx` のテスト2の `toHaveBeenCalledWith("/")` を `toHaveBeenCalledWith("/home")` に変えて流し、**失敗メッセージ**（期待と実際の差）を読む。読んだら元に戻す。
4. **`getBy` と `findBy` の違いを体感**：テスト3の `findByRole("alert")` を `getByRole("alert")` に変えて流し、なぜ失敗するか（待たないから）を確認する。元に戻す。
5. **権限テストを読む**：`test_backend.py` の `TestMemberViews` から「200・401・403」が揃う3テストを見つけ、それぞれ `login` の有無・種類がどう違うかを言葉で説明してみる。
6. **新テストを足す（バック）**：§15.2 を参考に、`TestModels` に「`def_choice` を作ると `def_symbol` が保存されている」ことを `assertEqual` で確かめるテストを1つ足し、流して緑になることを確認する。

---

18. この章のまとめ

- **テストは「機械にやらせる動作確認」**。健康診断のように、コードを直すたびに流して「壊していないか」を一瞬で確かめる、保守の最後の砦。
- どんなテストも **3ステップ**（準備 Arrange → 実行 Act → 検証 Assert）。検証＝**アサーション**（「〜のはず」を主張し、違えば失敗）。
- **フロント** は **Vitest ＋ React Testing Library**。`Login.test.tsx` 全132行を1行ずつ読み、`vi.mock`（画像・API・navigate のモック化）、`describe` / `it` / `beforeEach`、`render` / `screen` / `fireEvent` / `waitFor`、`getByRole` / `getByLabelText` / `findByRole`、`mockResolvedValueOnce` / `mockRejectedValueOnce`、そして **成功遷移・エラー表示・401遷移・別エラー** の各テストを完全に理解した。
- 要素取得は **`getBy`（最初からある）/ `findBy`（後から出る・待つ）/ `queryBy`（無いことの確認）** を使い分ける。
- **バック** は **Django `TestCase` ＋ `APIClient`**。`test_backend.py` の土台（`make_break_times`・`BaseTestCase`・`login` ヘルパー）と `TestModels` を1行ずつ読み、`assertEqual` / `assertIn` / `assertTrue` / `assertFalse` / `assertLessEqual` を理解した。
- APIテストの王道は **「未ログイン→401 / 正しい権限→200 / 権限不足→403」の3点セット**。401（誰か不明）と403（権限不足）の違いが肝。作成は201・削除は204も確認する。
- テストは **専用設定**（`test_settings.py`：使い捨てSQLite・即時実行・マイグレーション省略）で動かす。実行は **`npm run test:run`** と **`python manage.py test --settings=hozen_another.test_settings`**。`✓` / `OK` が出れば全合格。
- 失敗したら **メッセージ（期待と実際の差）を読み**、本体を直すかテストを更新するかを判断する。理解せずに消さない。
- **新テストは似た既存テストをコピーして一部を変える**。バグを直したら再発防止のテストを1つ足す。

これで本シリーズの本編は完了です。索引に戻り、必要な章をいつでも引けるようにしておきましょう。次は「README.md（ドキュメント索引へ戻る）」へ。

巻末の付録も使ってください:「このコードは何だっけ？」と思ったら「13_付録_出てくるコードとコマンドの一覧」（コードから意味を引く）、「こういう時はどう書くんだっけ？」と思ったら「14_付録_逆引き_やりたいことからコードを探す」（やりたいことから書き方を引く）を開きます。